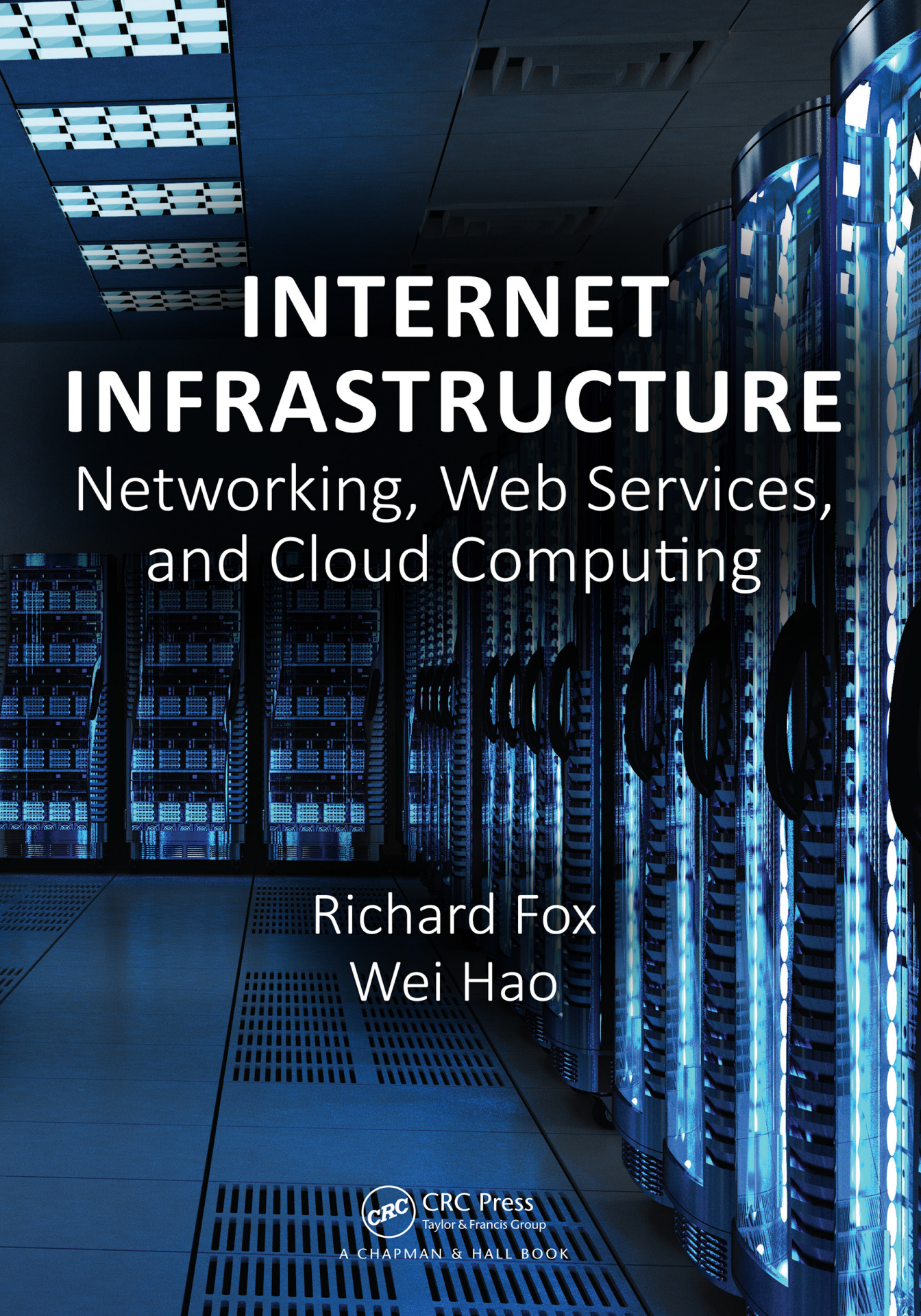




INTERNET INFRASTRUCTURE

Networking, Web Services,
and Cloud Computing

Richard Fox
Wei Hao



CRC Press
Taylor & Francis Group

A CHAPMAN & HALL BOOK

Internet Infrastructure



Taylor & Francis

Taylor & Francis Group

<http://taylorandfrancis.com>

Internet Infrastructure

Networking, Web Services, and Cloud Computing

by

Richard Fox

Department of Computer Science
Northern Kentucky University

Wei Hao

Department of Computer Science
Northern Kentucky University



CRC Press

Taylor & Francis Group

Boca Raton London New York

CRC Press is an imprint of the
Taylor & Francis Group, an **informa** business

A CHAPMAN & HALL BOOK

CRC Press
Taylor & Francis Group
6000 Broken Sound Parkway NW, Suite 300
Boca Raton, FL 33487-2742

© 2018 by Taylor & Francis Group, LLC
CRC Press is an imprint of Taylor & Francis Group, an Informa business

No claim to original U.S. Government works

Printed on acid-free paper

International Standard Book Number-13: 978-1-1380-3991-9 (Hardback)

This book contains information obtained from authentic and highly regarded sources. Reasonable efforts have been made to publish reliable data and information, but the author and publisher cannot assume responsibility for the validity of all materials or the consequences of their use. The authors and publishers have attempted to trace the copyright holders of all material reproduced in this publication and apologize to copyright holders if permission to publish in this form has not been obtained. If any copyright material has not been acknowledged please write and let us know so we may rectify in any future reprint.

Except as permitted under U.S. Copyright Law, no part of this book may be reprinted, reproduced, transmitted, or utilized in any form by any electronic, mechanical, or other means, now known or hereafter invented, including photocopying, micro-filming, and recording, or in any information storage or retrieval system, without written permission from the publishers.

For permission to photocopy or use material electronically from this work, please access www.copyright.com (<http://www.copyright.com/>) or contact the Copyright Clearance Center, Inc. (CCC), 222 Rosewood Drive, Danvers, MA 01923, 978-750-8400. CCC is a not-for-profit organization that provides licenses and registration for a variety of users. For organizations that have been granted a photocopy license by the CCC, a separate system of payment has been arranged.

Trademark Notice: Product or corporate names may be trademarks or registered trademarks, and are used only for identification and explanation without intent to infringe.

Library of Congress Cataloging-in-Publication Data

Names: Fox, Richard, 1964- author. | Hao, Wei, 1971- author.
Title: Internet infrastructure : networking, web services, and cloud computing / Richard Fox, Wei Hao.
Description: Boca Raton : Taylor & Francis, a CRC title, part of the Taylor & Francis imprint, a member of the Taylor & Francis Group, the academic division of T&F Informa, plc, [2017] | Includes bibliographical references and indexes.
Identifiers: LCCN 2017012425 | ISBN 9781138039919 (hardback : acid-free paper)
Subjects: LCSH: Internet. | Internetworking (Telecommunication) | Web services. | Cloud computing.
Classification: LCC TK5105.875.I57 F688 2017 | DDC 0040.678--dc23
LC record available at <https://lccn.loc.gov/2017012425>

Visit the Taylor & Francis Web site at
<http://www.taylorandfrancis.com>

and the CRC Press Web site at
<http://www.crcpress.com>

Dedication

Richard Fox dedicates this book to Cheri Klink, Brandon Kozloff, Danielle Kozloff, Tony Garbarini, and Joshua Smith, and in loving memory of Brian Garbarini.

Wei Hao dedicates this book to his parents, Yongsheng Hao and Rongfen Wang; his wife, Jiani Mao; and his children, Cody Hao and Sophie Hao.



Taylor & Francis

Taylor & Francis Group

<http://taylorandfrancis.com>

Contents

Acknowledgments.....	xv
Preface.....	xvii
Authors.....	xix

Chapter 1	An Introduction to Networks.....	1
1.1	Network Communication	1
1.1.1	Network Devices	2
1.1.2	Servers	5
1.1.3	Network Media.....	7
1.1.4	Network Hardware	11
1.2	Types of Networks	13
1.2.1	Network Topology	13
1.2.2	Classifications of Networks	16
1.3	Network Protocols	17
1.3.1	Transmission Control Protocol/Internet Protocol	18
1.3.2	Open Systems Interconnection.....	19
1.3.3	Bluetooth	22
1.3.4	Frame Relay	23
1.4	Ethernet	24
1.5	The Internet: An Introduction	26
1.5.1	What Is a Network Address?.....	26
1.5.2	Error Handling	28
1.5.3	Encryption Technologies.....	32
1.5.4	The Firewall	36
1.5.5	Network Caches	37
1.6	Chapter Review	39
	Review Questions	39
	Review Problems.....	40
	Discussion Questions	41
Chapter 2	Case Study: Building Local Area Networks	43
2.1	Introduction	43
2.2	Ethernet	45
2.2.1	Ethernet at the Physical Layer.....	45
2.2.2	Ethernet Data Link Layer Specifications	49
2.2.3	Building an Ethernet Local Area Network	51
2.3	Wireless Local Area Networks	53
2.3.1	Wireless Local Area Network Topologies and Associations	54
2.3.2	Wireless Local Area Network Standards.....	57
2.3.3	Wireless Hardware Devices	59
2.3.4	Wireless Local Area Network Frames.....	62
2.3.5	Setting Up a Wireless Local Area Network.....	65
2.3.6	Related Technologies.....	68
2.4	Securing Your Local Area Network.....	71
2.5	Virtual Private Networks.....	75

2.6	Chapter Review	77
	Review Questions	78
	Review Problems	78
	Discussion Questions	79
Chapter 3	Transmission Control Protocol/Internet Protocol	81
3.1	Introduction	81
3.2	Application Layer	82
3.2.1	File Transfer Protocol.....	83
3.2.2	Dynamic Host Configuration Protocol.....	85
3.2.3	Secure Sockets Layer and Transport Layer Security	87
3.2.4	Email Protocols	91
3.2.5	Secure Shell and Telnet	93
3.3	Transport Layer	94
3.3.1	Transmission Control Protocol Handshake and Connections	95
3.3.2	Datagrams: Transmission Control Protocol, User Datagram Protocol, and Others.....	96
3.3.3	Flow Control and Multiplexing	101
3.4	Internet Layer	102
3.4.1	Internet Protocol Version 4 and Internet Protocol Version 6 Addresses	102
3.4.2	Internet Protocol Version 4 Packets	107
3.4.3	Internet Protocol Version 6 Addresses.....	109
3.4.4	Establishing Internet Protocol Addresses: Statically and Dynamically	112
3.4.5	Internet Control Message Protocol and Internet Group Management Protocol	114
3.4.6	Network Address Translation.....	116
3.5	Link Layer	118
3.6	Chapter Review	121
	Review Questions	121
	Review Problems	122
	Discussion Questions	123
Chapter 4	Case Study: Transmission Control Protocol/Internet Protocol Tools	125
4.1	Packet Capture Programs	125
4.1.1	Wireshark	126
4.1.2	tcpdump	136
4.2	Netcat.....	137
4.3	Linux/Unix Network Programs.....	141
4.3.1	The Linux/Unix ip Command.....	142
4.3.2	Other Noteworthy Network Resources	143
4.3.3	Logging Programs.....	149
4.4	Domain Name System Commands	152
4.5	Base64 Encoding.....	156
4.6	Chapter Review	158
	Review Questions	158
	Review Problems	159
	Discussion Questions	160

Chapter 5	Domain Name System.....	161
5.1	Domain Name System Infrastructure	162
5.1.1	Domain Name System Client.....	162
5.1.2	Domain Name System Server	167
5.1.3	Domain Name System Databases	178
5.2	Domain Name System Protocol	186
5.3	Domain Name System Performance	189
5.3.1	Client-Side Domain Name System Caching	191
5.3.2	Server-Side Domain Name System Caching.....	194
5.3.3	Domain Name System Prefetching	196
5.3.4	Load Balancing and Domain Name System-Based Load Balancing.....	198
5.3.5	Client-Side Domain Name System versus Server-Side Domain Name System Load Balancing	209
5.4	Domain Name System-Based Content Distribution Networks	211
5.5	Domain Name System-Based Spam Prevention	213
5.6	Chapter Review	215
	Review Questions.....	215
	Review Problems.....	216
	Discussion Questions	217
Chapter 6	Case Study: BIND and DHCP	219
6.1	Bind	219
6.1.1	Installing BIND.....	219
6.1.2	Configuring BIND.....	221
6.1.3	Running the BIND Server.....	228
6.1.4	The rndc Utility	229
6.1.5	Simple BIND Configuration Example	230
6.1.6	Master and Slave BIND Configuration Example	235
6.1.7	Configuring Caching-Only and Forwarding DNS Servers	237
6.2	Dynamic Internet Protocol Addressing.....	243
6.2.1	Dynamic Host Configuration Protocol.....	243
6.2.2	ISC DHCP Server.....	245
6.2.3	Integrating the ISC DHCP Server with the BIND DNS Server.....	251
6.3	Configuring Dnssec for a Bind Server	252
6.4	Chapter Review	256
	Review Questions.....	257
	Review Problems.....	258
	Discussion Questions	259
Chapter 7	Introduction to Web Servers.....	261
7.1	Hypertext Transfer Protocol	262
7.1.1	How Hypertext Transfer Protocol Works.....	262
7.1.2	Hypertext Transfer Protocol Request and Response Messages	266
7.1.3	Cookies.....	271
7.2	Hypertext Transfer Protocol Secure and Building Digital Certificates	272
7.3	HTTP/2.....	276

7.4	Content Negotiation	281
7.4.1	Language Negotiation	282
7.4.2	Other Forms of Negotiation	283
7.5	Server-Side Includes and Scripts	285
7.5.1	Uses of Common Gateway Interface.....	286
7.5.2	Server-Side Includes	286
7.5.3	Server-Side Scripts	288
7.6	Other Web Server Features	290
7.6.1	Virtual Hosts	290
7.6.2	Cache Control.....	291
7.6.3	Authentication	292
7.6.4	Filtering	293
7.6.5	Forms of Redirection.....	294
7.7	Web Server Concerns	296
7.7.1	Backend Databases.....	297
7.7.2	Web Server Security.....	299
7.7.3	Load Balancing	302
7.8	Chapter Review	304
	Review Questions	304
	Review Problems	305
	Discussion Questions	306

Chapter 8 Case Study: The Apache Web Server..... 307

8.1	Installing and Running Apache.....	307
8.1.1	Installing an Apache Executable.....	307
8.1.2	Installing Apache from Source Code.....	308
8.1.3	Running Apache.....	311
8.2	Basic Apache Configuration.....	312
8.2.1	Loading Modules	313
8.2.2	Server Directives	315
8.2.3	Directory Containers.....	320
8.2.4	Access Files.....	325
8.2.5	Other Containers	326
8.2.6	Handlers	329
8.3	Modules	331
8.4	Advanced Configuration.....	333
8.4.1	Logging	333
8.4.2	Content Negotiation.....	337
8.4.3	Filters.....	341
8.4.4	Authentication and Handling Hypertext Transfer Protocol Secure	343
8.5	Other Useful Apache Features	345
8.5.1	Spell Checking	345
8.5.2	Controlling Headers	346
8.5.3	Virtual Hosts	349
8.5.4	Indexes Options	352
8.5.5	Controlling Caching.....	354
8.5.6	Efficiency Considerations.....	355

8.6	Redirection and Rewrite Rules.....	357
8.7	Executing Server-Side Scripts in Apache.....	361
8.8	Chapter Review	367
	Review Questions.....	367
	Review Problems.....	369
	Discussion Questions	370
Chapter 9	Web Caching	373
9.1	Introduction to the Cache	374
9.2	Cache Strategies	377
9.2.1	Cache Replacement Strategies	377
9.2.2	Cache Consistency	380
9.3	Cooperative Caching	383
9.4	Establishing a Web Proxy.....	388
9.4.1	Manual Proxy Setup.....	389
9.4.2	Proxy Auto Configuration	390
9.4.3	Web Cache Communication Protocol Interception.....	391
9.5	Dynamic Proxy Caching Techniques	398
9.5.1	Caching Partial Content of Dynamic Pages.....	398
9.5.2	Dynamic Content Caching Protocol	400
9.5.3	Internet Content Adaptation Protocol	403
9.5.4	Database Query Result Caching	404
9.6	Chapter Review	405
	Review Questions.....	405
	Review Problems.....	406
	Discussion Questions	409
Chapter 10	Case Study: The Squid Proxy Server	411
10.1	Introduction to Squid.....	412
10.2	Installing and Running Squid.....	412
10.3	Basic Squid Configuration.....	416
10.4	The Squid Caches.....	420
10.4.1	Squid File System Types	420
10.4.2	Configuring Squid Caches	422
10.5	Squid Neighbors	425
10.6	Access Control in Squid	434
10.6.1	The acl Directive	435
10.6.2	Example acl Statements.....	437
10.6.3	Access Control Directives	439
10.7	Other Squid Features.....	441
10.7.1	Squid Log Files	441
10.7.2	Redirection	445
10.7.3	Authentication Helpers.....	447
10.8	Chapter Review	449
	Review Questions.....	449
	Review Problems.....	450
	Discussion Problems	451

Chapter 11	Cloud Computing	453
11.1	Web System Qualities	454
11.1.1	Performance	454
11.1.2	Availability	458
11.2	Mechanisms to Ensure Availability	462
11.2.1	Redundant Array of Independent Disks	462
11.2.2	Redundant Array of Independent Network Interfaces	466
11.2.3	High-Availability Clustering	468
11.3	Scalability	469
11.3.1	Vertical Scaling	470
11.3.2	Horizontal Scaling	471
11.3.3	Auto Scaling	472
11.4	Cloud Computing	474
11.4.1	Cloud Characteristics	474
11.4.2	Cloud Deployment Models	478
11.5	Virtualization	481
11.5.1	Compute Virtualization	481
11.5.2	Storage Virtualization	482
11.5.3	Network Virtualization	487
11.6	Web Services	488
11.7	Chapter Review	491
	Review Questions	492
	Review Problems	493
	Discussion Questions	495
Chapter 12	Case Study: Amazon Web Services	497
12.1	Amazon Web Service Infrastructure	497
12.1.1	Global Infrastructure	497
12.1.2	Foundation Services	497
12.1.3	Platform Services	499
12.2	Using Amazon Web Service	500
12.2.1	Using Amazon Web Service through the Graphical User Interface	500
12.2.2	Using the Amazon Web Service Command Line Interface	501
12.3	Compute Service: Elastic Compute Cloud	503
12.3.1	Elastic Compute Cloud Concepts	503
12.3.2	Building a Virtual Server in the Cloud	506
12.3.3	Elastic Compute Cloud Storage Service	512
12.4	Amazon Web Service Network Service	526
12.4.1	Virtual Private Cloud	526
12.4.2	Route 53	531
12.5	Cloudwatch, Simple Notification Service, and Elastic Load Balancer	541
12.6	Establishing Scalability	549
12.7	Performance	552
12.7.1	ElastiCache	553
12.7.2	CloudFront	560

12.8	Security.....	563
12.9	Platform Services	570
12.9.1	Email through Simple Email Service.....	570
12.9.2	Relational Database Service	574
12.10	Deployment and Logging.....	577
12.10.1	CloudFormation.....	577
12.10.2	CloudTrail.....	583
12.11	Chapter Review	585
	Review Questions.....	585
	Review Problems.....	587
	Discussion Questions	588
	Bibliography	591
	Index.....	599



Taylor & Francis

Taylor & Francis Group

<http://taylorandfrancis.com>

Acknowledgments

First, we are indebted to Randi Cohen and Stan Wakefield for their encouragement and patience in writing and completing this book, which was several years in the making. We thank all of our students in CIT 436/536 who inspired this book. We are very grateful to Scot Cunningham for some very useful feedback on this text. We also thank Andy Greely for testing Web Cache Communication Protocol (WCCP) interception with Cisco routers and Squid proxy servers. We are also extremely grateful to the following two people who reviewed early drafts of the textbook and provided us insightful and useful comments: R. R. Brooks, Holcombe Department of Electrical and Computer Engineering, Clemson University, Clemson, South Carolina, and Robert M. Koretsky (Retired), The University of Portland, Portland, OR.



Taylor & Francis

Taylor & Francis Group

<http://taylorandfrancis.com>

Preface

Look around the textbook market and you will find countless books on computer networks, data communication, and the Internet. Why did we write this textbook? We generally see these books taking one of three forms. The first are the computer science and business-oriented texts that are heavy on networking theory and usage with little emphasis on practical matters. They cover Transmission Control Protocol/Internet Protocol (TCP/IP), Internet servers, and the foundations for telecommunications but do not provide guidance on how to implement a server. The second are the books that take the opposite approach: strictly hands-on texts with little to know the theory or foundational material. In teaching computer information technology courses, we have found numerous books that instruct students on how to configure a server but not on how the server actually works. Finally, there are books on socket-level programming.

This textbook attempts to combine the aspects of the first and second groups mentioned previously. We do so by dividing the material roughly into two categories: concept chapters and case study chapters. We present networks and the Internet from several perspectives: the underlying media, the protocols, the hardware, the servers and their uses. For many of the concepts covered, we follow them with case study chapters that examine how to install, configure, and secure a server that offers the given service discussed.

This textbook can be used in several different ways for a college course. As a one-semester introduction to computer networks, teachers might try to cover the first five chapters. These chapters introduce local area networks (LANs), wide area networks (WANs), wireless LANs, tools for exploring networks, and the domain name system. Such a course could also spotlight later topics such as the Hypertext Transfer Protocol (HTTP) and cloud computing. A two-semester sequence on networks could cover the entire book, although some of the case studies might be shortened if the course is targeting computer science or business students as that audience may not know servers such as Apache and Squid in depth. A computer information technology course might cover the case studies in detail while covering the concept chapters as more of an overview. Finally, an advanced networking course might cover Domain Name System (DNS) ([Chapter 5](#)), HTTP ([Chapter 7](#)), proxy servers ([Chapter 9](#)), and cloud computing ([Chapters 11 and 12](#)).

As we wrote this textbook, it evolved several times. Our original intention was to write a book that would directly support our course CIT 436/536 (Web Server Administration). As such, we were going to cover in detail the Bind DNS name server, the Apache web server, and the Squid proxy server. However, as we wrote this book, we realized that we should provide background on these servers by discussing DNS, Dynamic Host Configuration Protocol, HTTP, HTTP Secure, digital certificates and encryption, web caches, and the variety of protocols that support web caching. As we expanded the content of the text, we decided that we could also include introductory networking content as well as advanced Internet content, and thus, we added chapters on networks, LANs and WANs, TCP/IP, TCP/IP tools, cloud computing, and an examination of the Amazon Cloud Service.

The book grew to be too large. Therefore, to offset the cost of a longer textbook, we have identified the content that we feel could be moved out of the printed book and made electronically available via the textbook's companion website. Most of the items on the website are not optional reading but significant content that accompanies the chapters that it was taken from. You will find indicators throughout the book of additional readings that should be pursued.

In addition to the text on the website, there are a number of other useful resources for faculty and students alike. These include a complete laboratory manual for installing, configuring, securing, and experimenting with many of the servers discussed in the text, PowerPoint notes, animation tutorials to illustrate some of the concepts, two appendices, glossary of vocabulary terms, and complete input/output listings for the example Amazon cloud operations covered in [Chapter 12](#).

<https://www.crcpress.com/Internet-Infrastructure-Networking-Web-Services-and-Cloud-Computing/Fox-Hao/p/book/9781138039919>

Authors

Richard Fox is a professor of computer science at Northern Kentucky University (NKU), Newport, Kentucky, who regularly teaches courses in both computer science (artificial intelligence, computer systems, computer architecture, concepts of programming languages), and computer information technology (information technology fundamentals, Unix/Linux, Web Server Administration). Dr. Fox, who has been at NKU since 2001, is the current chair of NKU's University Curriculum Committee. Prior to NKU, Fox taught for 9 years at the University of Texas-Pan American, Edinburg, Texas. He has received two Teaching Excellence Awards from the University of Texas-Pan American in 2000 and from NKU in 2012, and NKU's University Service Award in 2016.

Dr. Fox received a PhD in computer and information sciences from The Ohio State University, Columbus, Ohio, in 1992. He also has an MS in computer and information sciences from The Ohio State University (1988) and a BS in computer science from the University of Missouri Rolla, Rolla, Missouri (now Missouri University of Science and Technology) (1986). This is Dr. Fox's third textbook (all three are available from CRC Press/Taylor & Francis Group). He is also the author or coauthor of over 50 peer-reviewed research articles primarily in the area of artificial intelligence.

Fox grew up in St. Louis, Missouri, and now lives in Cincinnati, Ohio. He is a big science fiction fan and progressive rock fan. As you will see in reading this text, his favorite composer is Frank Zappa.

Wei Hao is an associate professor of computer science at Northern Kentucky University (NKU), Newport, Kentucky. He came to NKU in August 2008 from Cisco Systems in San Jose, California, where he worked as a software engineer. He also worked for Motorola and Alcatel, where he co-invented a patent.

Wei has 38 peer-reviewed publications in scholarly journals and conference proceedings. His research interests include web technologies, cloud computing, and mobile computing. He won the Faculty Excellent Performance in Scholarly or Creative Activity Award from NKU in 2012. Dr. Hao teaches a wide variety of undergraduate and graduate courses in computer science and computer information technology, including cloud computing, Web Server Administration, storage administration, administrative scripting, network security, system architecture, computer networks, software testing, software engineering, iPhone programming, and advanced programming method.

Dr. Hao earned his PhD in computer science from the University of Texas at Dallas, Richardson, Texas, in 2007 with a specialization in web technologies.



Taylor & Francis

Taylor & Francis Group

<http://taylorandfrancis.com>

1 An Introduction to Networks

Everyone knows what the Internet is, right? We all use it, we rely on it, and our society has almost become dependent on it. However, do we really understand what the Internet is and how it works? To many, the Internet is some nebulous entity. It is *out there* and we *connect to it*, and messages magically traverse it. In this textbook, we explore the Internet and its many components.

This is not just another network textbook. Network textbooks have existed for decades. Many of them describe in detail the hardware and protocols that make up networks. Some are specific to just one protocol, Transmission Control Protocol/Internet Protocol (TCP/IP). Others explore how to write programs that we use on the network. Yet other books describe how to secure your network from attacks. This textbook has taken a different approach in exploring the Internet. We will cover the basics (networks in general, hardware, and TCP/IP), but then, we will explore the significant protocols that we use to make the Internet work. Using several case studies, we will examine the most popular software that help support aspects of the Internet: TCP/IP tools, a Domain Name System (DNS) server, a Dynamic Host Configuration Protocol (DHCP) server, a web server, a proxy server, web caching, load balancing, and cloud computing software.

In this chapter, we will start with the basics. We will first explore network hardware and some of the more popular network protocols (excluding TCP/IP). We will also look at several network-related topics such as error detection and correction, encryption, and network caches. Most of this material (and TCP/IP, covered in [Chapter 3](#)) set the stage for the rest of the textbook. So, sit back, relax, and learn about one of the most significant technologies on the planet.

1.1 NETWORK COMMUNICATION

Let us start with some basics. A *network* is a group of connected things. A *computer network* is a collection of connected computer resources. These resources include but are not limited to computers of all types, network devices, devices such as printers and optical disc towers, MODEMs (MODEM stands for MODulation DEModulation), the cable by which these resources are connected, and, of course, people. Most computers connected to a network are personal computers and laptops, but there are also servers, mainframe computers, and supercomputers. More recently, mobile devices such as smart phones and tablets have become part of computer networks. We can also include devices that are not general-purpose computers but still access networks, such as smart televisions (TVs), Global Positioning System (GPS) devices, sensors, and game consoles. [Figure 1.1](#) illustrates a network of computers connected by two network devices. In the figure, there are numerous computers and a server (upper right-hand corner) as well as two printers connected to two routers, which connect the rest of these devices to the Internet with a firewall set between the network and the Internet. In [Sections 1.1.1](#) through [1.1.4](#), we further define some of these terms.

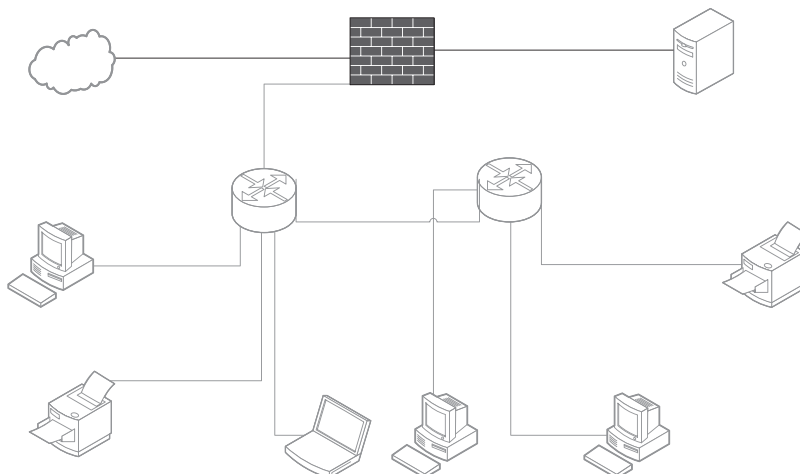


FIGURE 1.1 Example computer network.

1.1.1 NETWORK DEVICES

A *network device* is a device that receives a message from one resource on a network and determines how to pass the message along the network. The network device might directly connect to the destination resource, or it may connect to another network device, in which case it forwards messages on to the next device. Common network devices are hubs, switches, routers, and gateways. These devices can be wired, wireless, or both.

The *hub* is the most primitive of the network devices. It operates by receiving a message and passing it on to all the resources it connects to. The hub is sometimes referred to as a *multiport repeater*, because its job is to repeat the incoming message across all its ports (connections). Note that this is not the same as a multicast, which we will discuss later in this section.

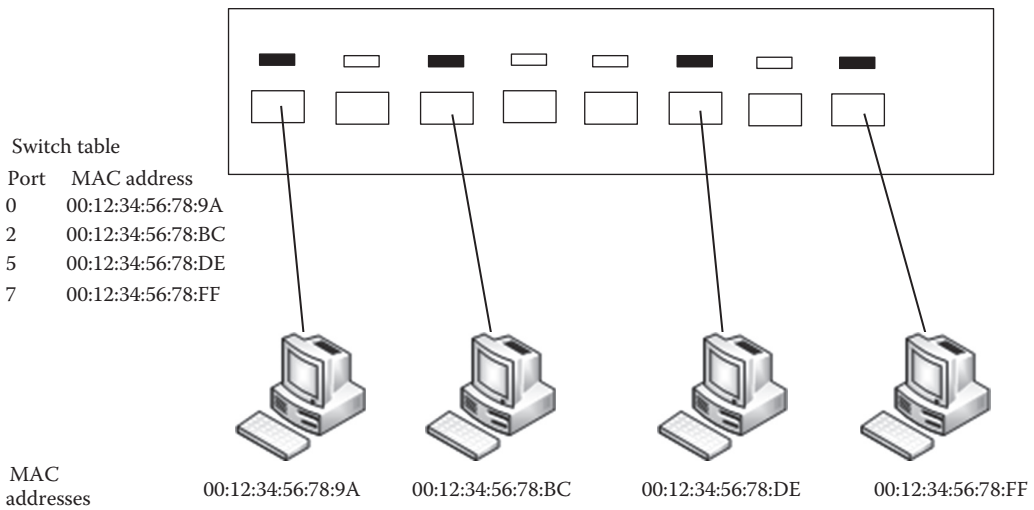
The hub also handles collision detection by forwarding a *jam signal* to all the connected devices, should multiple messages arrive at the same time. The jam signal indicates that a message collision occurred among the devices connected to it. When this happens, each device trying to communicate waits for a random amount of time before retrying to resend its message. Hubs are mostly obsolete today because of superior devices such as the network switch.

The network *switch* passes an incoming message onto a single resource. The switch uses the message's destination address to determine the device to which the message should be passed. This address is known as a low-level address and is referred to as the *hardware* address or the *media access control* (MAC) address. The switch is also known as a *MAC bridge*.

When a device is connected to a switch, the switch acquires that device's MAC address and retains it in a table. This table is a simple listing that for each port on the switch, the attached device's hardware address is stored. In [Figure 1.2](#), we see a switch connecting four devices and the table that the switch maintains. Notice that since the switch has more than four ports, some of the port numbers are currently not used in the table.

On receiving a message, the switch examines the destination MAC address and forwards the message on to the corresponding port, as specified in its table. Some switches can also operate on network address (e.g., IP addresses). The main difference between a switch and a router is that the router operates on network addresses exclusively and not on hardware addresses. We will differentiate between types of switches later in this chapter.

The *router* operates at a higher level of the network protocol stack than the switch. The router utilizes the message's destination *network* address to route the message on to its next step through the network. This network address is dependent on the type of network protocol. Assuming TCP/IP,

**FIGURE 1.2** Network switch and its table.**TABLE 1.1**
Sample Routing Table

Network Destination	Netmask	Gateway	Interface	Metric
0.0.0.0	0.0.0.0	10.15.8.1	10.15.8.164	10
10.15.8.0	255.255.252.0	On-link	10.15.8.164	266
10.15.8.164	255.255.255.255	On-link	10.15.8.164	266
127.0.0.0	255.0.0.0	127.0.0.1	127.0.0.1	306
192.168.56.0	255.255.255.0	192.168.56.1	192.168.56.1	276
192.168.56.1	255.255.255.255	192.168.0.100	192.168.56.1	276
192.168.0.100	255.255.255.255	127.0.0.1	127.0.0.1	306
224.0.0.0	240.0.0.0	On-link	192.168.56.1	276
255.255.255.255	255.255.255.255	On-link	10.15.8.164	266

the network address is an Internet Protocol version 4 (IPv4) or Internet Protocol version 6 (IPv6) address. The *next step* does not necessarily mean the destination device. Routers route messages across networks, so that they are forwarded on to the next point in the network that takes the message closer to its destination. This might be to the destination computer, to a network switch, or to another router. Routers therefore perform *forwarding*. A sample network routing table is shown in [Table 1.1](#) (the content of the routing table, including terms such as netmask and interface, is discussed later in this chapter). Metric is a cost of using the indicated route. This value is used by the router to determine the *hop* that the message should take next, as it moves across the network.

The *gateway* is a router that connects different *types* of networks together. More specifically, the gateway has the ability to translate a message from one protocol into another. This is handled by hardware or software that maps the message's nondata content from the source network's protocol to the destination network's protocol. [Figure 1.3](#) shows two different types of local area networks (LANs) connected by a gateway. The gateway is like a router, except that it is positioned at the *edge* of a network. Within a LAN, resources are connected by routers or switches. Routers and gateways connect LANs together. Oftentimes, a LAN's connection to the Internet is made through a gateway rather than a router.

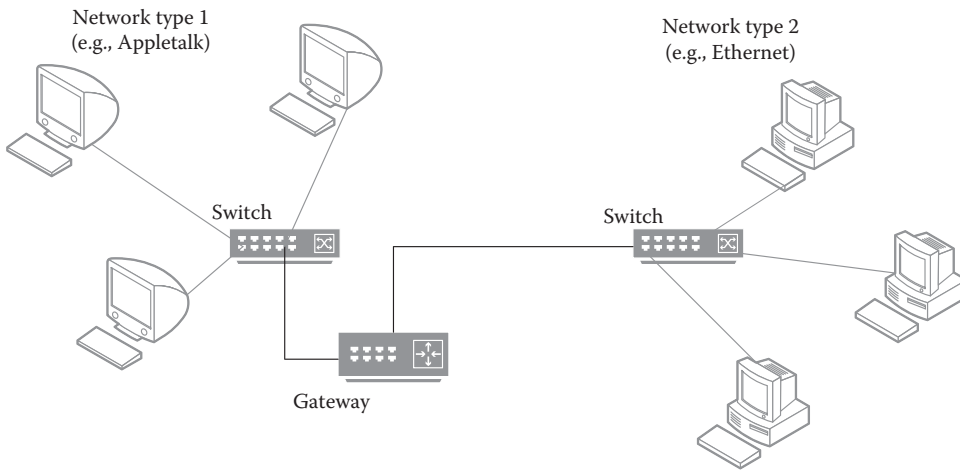


FIGURE 1.3 Positioning the gateway at the *Edge* of networks.

Note that the terms switch, router, and gateway are sometimes used interchangeably. For instance, switches that also utilize IP addresses are sometimes referred to as *layer 3 switches*, even though they are operating like routers. Routers are sometimes referred to as gateways whether they are translating protocols or not. As stated previously, we will visit protocols later in this chapter, and at that time, we will revisit the roles of the switches, routers, and gateways.

The typical form of communication over a network is a *unicast*. This form of communication allows a message to be sent from one source device to one destination device. The source and destination will typically open a communication channel (session) where communication may be one-directional or bi-directional (in which case, it is known as a duplex mode). However, there are times when communication is a one-to-many or many-to-many situation. This occurs when one or more devices are communicating with multiple devices. That is, there are multiple destination devices that a message is intended for. Such a communication is known as a *multicast*. A hub performs a limited form of multicast. A more specific reason for a multicast occurs when a server is streaming content to many destinations. Rather than duplicating the message at the server end, the message is sent out onto the network where routers are responsible not just for forwarding the content but also duplicating the content to be sent to multiple destinations. Another example for a multicast is with a multiplayer networked computer game. When one player performs an operation from within the software, all other players must see that move. The player's computer does not have to duplicate messages to send to all other players. Instead, the routers take care of this by duplicating the message, resulting in a multicast.

Two other forms of communication are *broadcast* and *anycast*. A broadcast is a message sent from one device to all others on its local subnetwork (we define a subnet later in this chapter). The hub is a network broadcast device in that its job is to broadcast to all devices on its local network. Although this is like a multicast in that a message is duplicated, it is a multicast within a very limited setting. In other words, a multicast is a broadcast where destinations are not restricted to the local subnetwork. Finally, an anycast is somewhat of a compromise between a unicast and a multicast. With an anycast, there are several destinations that share the same IP address. A message is sent that could conceivably go to any of these destinations but is always routed to the nearest destination. In this way, an anycast will reach its destination in the shortest amount of time. We will refer to multicast, broadcast, and anycast from time to time through the text. If we do not explicitly mention the form of communication, assume that it is a unicast.

Figure 1.4 illustrates the difference between unicast, multicast, anycast, and broadcast. In this subnetwork, six devices are connected to our network device (a switch in this case). On the left,

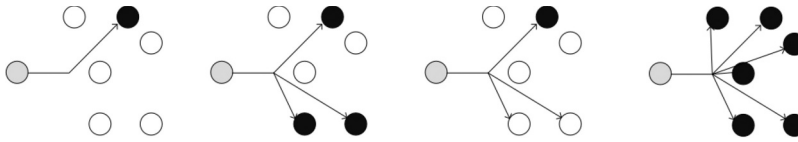


FIGURE 1.4 Comparing unicast, multicast, anycast, and broadcast.

we have a unicast message, in which the switch sends the message to a single device. Next, we have a multicast, in which the switch sends the message to several specified devices. Then, we have anycast, in which the switch sends a message to all devices with the same IP address, but only one needs to receive it. Finally, on the right, the message is broadcast to all devices.

1.1.2 SERVERS

The word *server* can be used to describe either hardware or software. Many users will refer to the server as a physical device, because they associate a specific computer with the service task (such as a file server or a web server). However, in fact, the server is a combination of the physical device designated to handle the service request and the server software package. By referring to the physical device as a server, we are implying that the device is not used for any other purpose. This may or may not be true. For instance, a computer may be set up as a web server but may also be someone's personal computer that he or she uses for ordinary computing. On the other hand, the server software is essential, because without it, the hardware device is merely a computing device. Therefore, we need to install, configure, and run the server software on the computer. In most cases, we will refer to servers as the software, as we are mostly interested in exploring how the server software works.

The use of servers in a network suggests that the network is set up as using a *client-server model*. In such a network, most resources are clients, which make requests of the server(s) of the network. A server's role within such a network is to service client requests. The other form of network is known as a *peer-to-peer* network, where there are no servers, and so, all resources are considered equal. We might view the peer-to-peer network as one where any resource can handle the given request or where none of the clients make requests of other resources.

There are many types of servers, and no doubt, you must have heard of, interacted with, or possibly even installed several of these servers. This book discusses several of them and covers, in detail, how to set up, configure, secure, and monitor a variety of popular servers. [Table 1.2](#) presents a list of network servers. This is not meant to be a complete list, but we will discuss many of these over the course of this book.

Cloud computing has brought about a change in how organizations provide network services. Instead of physically housing servers within their own domain, many organizations are purchasing space and processing power from companies that support such activities via the cloud. In this way, the organization does not need to purchase, install, or maintain the hardware. This is all handled by the company that they are purchasing access to. A company's cloud is a collection of servers (sometimes called a server farm) that run *virtual machines* (VMs). A VM is a combination of hardware, software, and data, whereby a server emulates a dedicated computer of some platform, made available via network to the end user(s) of the VM. The VM hardware is the server. The VM software is the program that performs the emulation. The VM data is the environment that the user has established, such as installed software, configuration files, and so forth. Companies like Amazon and Microsoft can support VMs of hundreds or thousands of different clients. In many cases, the VMs are not of simple end-user computers but of servers such as web servers and proxy servers. We cover cloud computing in [Chapter 11](#) and look at Amazon Web Services (AWSs) in [Chapter 12](#).

TABLE 1.2
Types of Network Servers

Server Type	Usage/Role	Examples
Application server	Runs application software across the network, so that applications do not need to be installed on individual computers. In older client–server networks, an application server was often used so that the clients could be diskless (without a hard disk) and thus reduce the cost of the clients.	ColdFusion, Enterprise Server, GlassFish, NetWeaver, Tomcat, WebLogic, WebSphere, Windows Server
Database server	Provides networked access to a backend database.	DB2, Informix, Microsoft SQL Server, MySQL, Oracle
DHCP server	Provides dynamic IP addresses to clients in a network.	DHCP Server, dnsmasq, ISC DHCP
Email server	Transfers email messages between local client and email servers; gives clients access to incoming email (email messages can be stored on the server, client, or both).	Apache James, IBM Lotus Domino, Microsoft Exchange Server, Novell Groupmail, Novell NetMail, OpenSMTPD, Postfix/Sendmail
File server	Shared storage utility accessible over the network; there are many subclasses of file servers such as FTP servers, database servers, application servers, and web servers.	See the more specific types of servers
FTP server	Supports File Transfer Protocol, so that clients can upload and download files to and from the server. FTP is insecure, so it is not commonly used, except for possible anonymous file transfers (permitting clients to transfer public files without logging in). SFTP is FTP over an SSH connection, whereas FTPS is FTP made secure by using an SSL connection (SSL is discussed in Chapter 3).	Cerberus FTP Server, FileZilla Server, ftpd, freeFTPD, Microsoft Internet Information Services, WS FTP
Gaming server	Same as an application server, except that it is dedicated to running a single game to potentially thousands or millions of users across the Internet.	Varies by game
List server	Type of mail server that manages lists, so that email messages can be sent to all on the list. This feature may be built into the mail server.	LISTSERV, Mailman, Sympa
Name server (or DNS server)	Resolves domain names (IP aliases) into IP addresses and/or serves as a cache.	BIND, Cisco Network Registrar, dnsmasq, Simple DNS Plus, PowerDNS
Print server	Connects clients to a printer to maintain a queue of print jobs, as submitted by the clients. Provides clients with feedback on status of print jobs.	CUPS, JetDirect, NetBOIS, NetWare
Proxy server	Intermediaries between web clients and web servers. The proxy server has several roles: caches web pages within an organization for more efficient recall; provides security mechanisms to prevent unwanted requests or responses; and provides some anonymity for the web clients. A reverse proxy server is used as a front end to a web server for load balancing.	CC Proxy Server, Internet Security and Acceleration Server, Squid, WinGate
SSH server	Uses the Secure Shell Protocol to accept connections from remote computers. Note that FTP does not permit encryption, but secure versions are available over SSH such as SFTP.	Apache MINA SSHD, Copssh, OpenSSH, Pragma Systems SSH Server, Tectia SSH Server

(Continued)

TABLE 1.2 (Continued)
Types of Network Servers

Server Type	Usage/Role	Examples
VM server	Server that creates virtual machines that themselves can be used as computers, web servers, email servers, database servers, and so on.	Microsoft Hyper-V, VMware vSphere Server
Web server	Form of file server to respond to HTTP requests and return web pages. If web pages have server-side scripts, these are executed by the web server, making it both a file server and an application server.	AOLserver, Apache, Internet Information Services, NGINX, OpenLiteSpeed, Oracle HTTP Server, Oracle WebLogic Server, TUX web server

1.1.3 NETWORK MEDIA

We need to connect the components of a computer network together. The connection can come in two different forms: wired and wireless. We refer to these connections as the *media*. We can rate the efficiency of a medium by its *bandwidth*, which is the amount of data that can be carried over the medium in a unit of time, such as bits per second.

The wired forms of network connectivity are twisted-wire pair (or just twisted pair), coaxial cable, and fiber optic cable. The earliest form of twisted-wire pair was used in telephone lines, and it is the oldest and most primitive form. It has also been the cheapest. Although enhancements have been made with twisted-wire technology to improve its bandwidth, it still has the lowest bandwidth among the wired forms. In many cases, use of both coaxial cable and fiber optic cable have replaced the twisted-wire cable for longer-distance computer networks.

Both twisted-wire pair and coaxial cable contain a piece of wire over which electrical current can be carried. For the wire to carry current, the wire must be both whole and connect to something at the other end. That is, the wire makes up a closed circuit. At one end, a voltage source (e.g., a battery) is introduced, and the current is said to flow down the wire to the other end. Although any metals can be used as a conductor of electrical current, we typically use copper wire, which is found in both twisted-wire pair and coaxial cable.

Figure 1.5 illustrates the three forms of cable. On the left, there are four strands of twisted-wire pair, placed into one cable. In the middle, there is a coaxial cable, with the cable extruding from the end. Surrounding the cable are layers of insulation. On the right, there are *several thousand* individual pieces of fiber optic cable. Each individual cable is flexible, as you might notice from the entire collection of cables bending.



FIGURE 1.5 Forms of cable. (Reprinted from Fox, R., *Information Technology: An Introduction for Today's Digital World*, CRC Press, Boston, FL, 2013. With permission.)

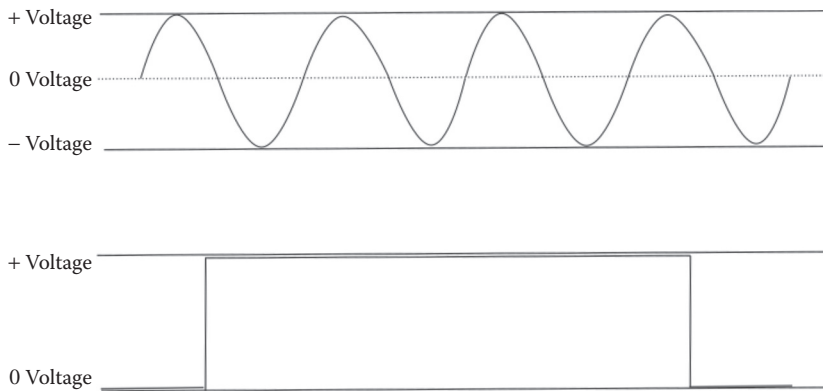


FIGURE 1.6 AC (top) and DC (bottom).

Current flowing down a wire can be in one of two forms. Current that comes out of our power sockets is typically in the form of direct current (DC). Direct current means that the current flows at a steady voltage. For telecommunications, we want to use alternating current (AC). Alternating current looks something like a waveform, which is a series of oscillations between peaks and values. [Figure 1.6](#) illustrates the difference between DC and AC. Because our power supply is often in the form of DC, a computer needs to convert from DC to AC when using a wired network.

With DC, voltage is the only property of interest. However, with AC, there are a number of different values associated with the current flowing down the wire: amplitude, frequency, period, phase, and wavelength. Amplitude defines the height of the peak (the Y-axis value at its height). The frequency defines the number of waves (cycles) per second. We express this value by using the term *hertz* (or kilohertz, megahertz, etc.), where 1 hertz means one full cycle per second and a kilohertz is 1000 cycles per second. The period is the amount of time for one full wave or the starting point to a peak to a valley and back. This denotes the time along the X-axis that it takes for the wave to go from its starting point on the Y-axis to the same location after having reached both a peak and a valley. The frequency and the period are reciprocals of each other, or $\text{period} = 1/\text{frequency}$ and $\text{frequency} = 1/\text{period}$. The term *phase* applies to multiple waveforms combined together, which can occur, for instance, when multiplexing a signal. Finally, wavelength is the same as period multiplied by the speed of sound. For network purposes, we will limit our interest to just frequency and phase.

Early wire that was used to conduct electricity for telecommunication purposes (e.g., telegraph and telephones) suffered from noise caused by electromagnetic interference from various sources, including other wire. The advent of twisted-wire pair allowed signals to be carried with greatly reduced interference. The current flowing down two wires in parallel causes two canceling magnetic fields to protect the content being carried down the wires. Twisted-wire pair quickly became the basis for the public telephone network and eventually was used for computer networking.

The shielded forms are still commonly in use for the *landline* telephone network, whereas the unshielded forms can be found in Ethernet-style LANs. Advantages of the twisted-wire pair are as follows: it is thin and flexible; multiple twisted-wire pairs can be grouped together into one supporting cable (dozens, hundreds, and even thousands); the cable is cheap to produce; and cross-talk is minimized. The primary disadvantage is its limited bandwidth, but it also suffers from a limited maximum cable length, requiring that repeaters be placed between lengths of cable.

Coaxial cable consists of a copper conducting cable placed inside flexible insulating material, which itself is surrounded by another conducting layer (foil or copper braid). The inner cable and the outer layer provide a circuit, whereas the outer layer also acts as a shield to reduce outside interface. This construction is placed into a somewhat-flexible outer shell. The coaxial cable offers a higher bandwidth over most forms of twisted-wire pair, and a longer length of cable (over twisted-wire pair) is possible and thus requires fewer repeaters. The disadvantage of coaxial cable is that it is more expensive than

the twisted-wire pair. It is also less flexible and thicker. Obviously, coaxial cable has found great use in below-ground cabling, delivering cable televisions to households across the world. Coaxial cable is also an attractive option for LAN cabling if one does not mind a slightly greater expense.

Fiber optic cable does not rely on electrical current at all, but instead, it is a piece of glass. Information is sent over fiber optic cable by transmitting pulses of light. This has a number of advantages over wire-based forms of communication. Before we look at this, let us examine fiber optic cable more closely.

The cable is made up of one or more optical *fibers*, each of which is a piece of extruded glass (or possibly plastic). Each fiber is around 125 microns in diameter (around 0.005 of an inch). A human hair, by comparison, is no more than 180 microns and is often smaller (ironically, a human hair can also be used like an optical fiber to transmit light). The fiber consists of three parts. At the center of the fiber, the core is present, which is capable of transmitting light pulses at different frequencies and along great distances. Further, the material is not refracting, meaning that the light's wavelength should not change as the light travels. The core is surrounded by what is called *cladding material*, a coating that reflects any light back into the fiber. Outside of the cladding, a plastic coating is present to protect the fiber from damage or moisture. A bundle, a single cable, can then consist of any number of fibers, perhaps, as many as thousands of individual fibers. Cores generally come in two forms, single-mode and multi-mode. The single-mode core transmits infrared laser light of a greater wavelength, whereas multi-mode core transmits infrared light of a lower wavelength.

The fiber optic cable has numerous advantages over copper wire. These are listed as follows:

- Far greater bandwidth in that a single optical fiber can carry as much as 111 gigabits per second with a typical bandwidth of 10 to 40 gigabits per second.
- Far greater distance at which signals can travel without loss or need of repeaters.
- Far less susceptibility to outside influences such as radio noise or lightning strikes. Fiber optic cable is not affected by electromagnetic radiation unlike copper wire (whether shielded and insulated or not).
- Optical cable is lighter in weight and is safer because it is nonconducting.
- There is less risk of theft. This last point is an odd one to make. However, there are many *copper thieves* in society today, looking to steal sources of copper and sell them. As fiber optic cable is not made of copper, it poses a far less risk of being stolen despite the fiber being a more expensive item to produce.

The main drawback of fiber optic cable is that it is far more expensive than copper cable. An even greater expense is that of replacing millions of miles of twisted-wire pair of our public telephone network with fiber optic cable. Many telecommunication companies have invested billions of dollars to do just that to upgrade the telephone networks and provide broadband Internet access to households. However, this cost is one that is being shared among all users (for instance, this is part of the price that you pay to receive Internet access at home, whether your household previously had twisted-wire pair or not). Fiber optic cable is more brittle than copper wire and more difficult to repair if the length of cable is cut.

Unlike copper wire, which transmits analog information (e.g., sounds), optical fiber directly transmits digital information (1s and 0s sent as pulses of light). Thus, there is less work required at either end of the transmission to send or receive computer data. For copper wire, information might first need to be *modulated* (converted from a digital form to an analog form) for transmission and then *demodulated* (converted from an analog form back to a digital form) on receipt. The MODEM is a device that performs these two tasks.

The other form of media is wireless communication. As its name implies, this form of communication does not involve any form of cable. Instead, information is converted into a waveform and broadcast through the air. Usually, the waveform is a radio frequency (RF) or high-frequency

wave (microwave); however, it could also be an infrared signal. Wireless communication is far more limited in the distance at which a message can travel, and so, there needs to be a nearby *wireless access point*. For a person's home, the wireless access point is most likely a wireless hub, a router, or a MODEM, which then connects to a wired communication line.

Wireless communication is also less reliable than wired forms in that there can be many things that interfere with the signal: a door or wall, devices that emit their own radio signals, or nearby electrical devices. In addition, wireless communication requires additional security mechanisms to avoid someone from easily *eavesdropping* on the communication. Encryption is commonly used today to reduce this concern, but it is only valuable if the users know how to employ it. Other best practices are also available to secure wireless communication, such as metallic paint on exterior walls to limit the amount of *leakage* of the signal getting outside of the building. Finally, speed of wireless communication lags behind that of wired forms. On the other hand, there is far less cost in using wireless communication because there is no cable to lay (possibly below ground) while offering greater convenience in that a computer user is not tied to one location.

Let us examine bandwidth more formally. This term defines the amount of data that can be transmitted at a time. Formally, bandwidth is given in terms of *hertz*, as introduced earlier. This term conveys the fraction of a second that it takes for some quantity of transmission, such as 1 bit. However, for convenience, we usually refer to bandwidth as the number of bits transmitted per second. Table 1.3 provides a comparison of the bandwidth capabilities of the forms of transmission discussed previously. You can see that fiber optic cable provides the greatest bandwidth, while wireless using radio frequencies provides the least bandwidth.

Let us try to put bandwidth in perspective and within a historical context. A small colored jpg image will be somewhere around 1 MByte in size, which is 8 Mbits or roughly 8 million bits. At a bandwidth of 1 bit per second, it would take 8 million seconds to transfer this file across a network (92 ½ days). Of course, all bandwidths today are far greater than 1 bit per second. At the maximum theoretical bandwidth for fiber optic cable, the transmission will take 0.00008 seconds.

In the mid to late 1970s, when personal computers were first being pioneered, most forms of telecommunication used the telephone network (which used twisted-wire pair), requiring that the home computer user access the telephone network by a MODEM. Original MODEMs had a baud rate of 300 bits per second. Baud rate is not the same as bandwidth, as baud rate defines the number of changes that can occur per second. A change will be based on the size of the unit being transmitted. For instance, if we are transmitting not bits but 2 bit *symbols*, then the baud rate will be twice that of the bandwidth. However, for the sake of this historical comparison, we will consider them to be equivalent.

By the early 1980s, baud rate had improved to 1200 bits per second, and by the mid-1980s, it was up to 4800 and then 9600 bits per second. In the 1990s, speeds increased again to 14,400, 28,800, and then final 56,000 bits per second (which is the theoretical limit for signals carried over the analog telephone network). As we saw in Table 1.3, both shielded twisted-wire and coaxial cables have

TABLE 1.3
Network Media Bandwidths

Type	Minimum Bandwidth	Maximum (Theoretical) Bandwidth
Shielded twisted-wire pair	10 Mbps	100 Mbps
Unshielded twisted-wire pair	10 Mbps	1000 Mbps
Coaxial cable	10 Mbps	100 Mbps
Fiber optic cable	100 Mbps	100 Gbps
Wireless using radio frequency	9 Kbps	54 Mbps

TABLE 1.4
Download Times for Various Bandwidths

Medium	Download Time
MODEM (300 bps)	7 hours, 24 minutes, 27 seconds
MODEM (1200 bps)	1 hour, 51 minutes, 7 seconds
MODEM (9600 bps)	13 minutes, 53 seconds
MODEM (14,400 bps)	9 minutes, 16 seconds
MODEM (28,800 bps)	4 minutes, 38 seconds
MODEM (56,000 bps)	2 minutes, 23 seconds
MODEM (300 mbps)	0.027 seconds
Twisted-wire and coaxial cable—minimum/maximum	0.8 second/0.08 second
Twisted-wire (unshielded)—maximum	0.008 second
Fiber optic—minimum/maximum	0.08 second/0.00008 second
3G	0.67 second
4G	0.08 second
Wireless (802.11n)	0.013 second

bandwidths that range between 10 Mbps (million bits per second) and 100 Mbps, and the bandwidth of fiber optic cable ranges between 100 Mbps and 100 Gbps (billion bits per second). We can add to this three additional forms of communication, all of them wireless. The minimum speed for the 3G cellular network was 200 Kbps (thousand bits per second), and the minimum speed for the 4G cellular network is estimated at 100 Mbps, whereas wireless communication through a wireless network card is estimated to be between 11 Mbps (802.11b) and 600 Mbps (802.11n). [Table 1.4](#) demonstrates the transfer time for each of the media discussed in this paragraph and the last paragraph for our 1 MB image file.

1.1.4 NETWORK HARDWARE

We have already discussed network devices that *glue* a network together. However, how does an individual computer gain access to the network? This is done by one of two devices, a network interface card (NIC) or a MODEM. Here, we explore what these two devices are.

An NIC is used to connect a computer to a computer network. There are different types of NICs depending on the type of network being utilized. For instance, an Ethernet card is used to connect a computer to an Ethernet network, whereas a Token Ring Adapter is used to connect a computer to a Token Ring network. As the most common form of local network today is an Ethernet, we typically use Ethernet cards as NICs.

The NIC is an electronic circuit board that slides into one of a computer's expansion slots on the motherboard. The card itself at a minimum will contain the circuitry necessary to communicate over the network. Thus, the network card must have the means of converting the binary data stored in the computer to style of information needed for the network media (e.g., electronic signals or light pulses). In addition, the data sent out onto the network must be converted from sequences of bytes (or words) into sequences of bits. That is, the NIC receives data from memory as words or bytes and then sends out the bits sequentially or receives bits sequentially from the network and compiles them back into bytes and words to be sent to memory.

With the greatly reduced cost of manufacturing electronic components, NICs have improved dramatically over time. Today, NICs will include many other features such as interrupt handling, direct memory access to move data directly to or from memory without CPU intervention, and collision-handling mechanisms, to name a few. In addition, modern NICs often have multiple buffers to store

incoming or outgoing packets so that partial messages can be stored until the full message arrives. The NICs can also handle some simple communication tasks such as confirming receipt of a message.

The NIC will have its own unique hardware address, commonly known as a MAC address. The NICs were manufactured with these addresses, but today, these addresses can be altered by the operating system or assigned randomly. Two reasons to permit the change in MAC addresses are to improve security (to avoid MAC address spoofing) and to support network virtualization. Computers may have multiple NICs if this is desirable. In such cases, the computer is given multiple, distinct addresses (e.g., multiple IP addresses).

The NIC has a port or a connection that sticks out of the computer. The network is physically connected to this port through some form of plug. Many types of plugs are used, but the most common used today is known as an RJ-45 connector, which looks like a telephone plug (RJ-11 connector) but is slightly larger (the RJ-45 has four pairs of wires, whereas the RJ-11 has three pairs of wires, and so, the RJ-45 connector is larger).

Many newer NICs can also handle wireless communication. A wireless NIC is one that sends radio signals to a nearby device called an access point. The access point connects to a wired portion of the network. Thus, the NIC communicates to the access point, which then communicates with a switch or router to other resources.

The radio signal that the NIC transmits can be one of four frequencies. Most common is a narrowband high-radio-frequency signal. The radio signal is limited in distance to usually no more than a few thousand feet before the signal degrades to the point that there is data loss. Two alternatives are to use either infrared light or laser light. These two approaches require line-of-sight communication. That is, the device housing the NIC must be in a clear line of sight with the access point. This is a drawback that restricts movement of the device so that it is seldom used for computers but may very well be used for stationary devices or when devices are in much closer proximity (a few feet), such as a television remote control.

The term *MODEM* stands for *MOD*ulation *DEM*odulation. These two terms express the conversion of digital information into analog information (modulation) and analog information into digital information (demodulation). These two conversion operations are necessary if we wish to communicate binary information (computer data) over an analog medium such as the public telephone network. The MODEM would connect to your telephone line in some fashion, and before the data could be transmitted, it would be modulated. Specifically, the binary data would be combined into a tone, and the tone would be broadcast over the analog phone lines. At the receiving end, the tone would be heard by a MODEM and demodulated back into the binary data.

The problem with using a MODEM is the reliance on the relatively low bandwidth available over the public telephone network. Original MODEM speeds were limited to 300 bits per second. Over a couple of decades, speeds improved but reached a maximum speed of 56,000 bits per second, because the telephone lines forced this limitation. The reason for this restriction is that the telephone lines were set up to carry a range of frequencies slightly greater than that of human hearing. Thus, the tones that the MODEM can broadcast are restricted to be within that range. Since there are a limited number of tones that the MODEM can broadcast, there is a limited amount of data that can be transmitted per tone.

Before broadband communication was readily available in people's households, another option to avoid the low-bandwidth telephone lines was to use a direct line into the household, known as a digital subscriber loop (DSL), or the coaxial cable line into your home that you use to carry cable television signals. DSL still used the telephone network, but the dedicated line provided a much greater bandwidth than the 56K mentioned previously. To communicate over the cable television wire, you would need a type of MODEM called a cable modem. Today, cable modems can achieve broadband speeds and are no longer required to perform modulation and demodulation, so the name is somewhat outdated. DSL is still used where broadband is unavailable, such as in rural areas.

As noted in [Table 1.4](#), we prefer higher bandwidths than what the MODEM over the telephone lines or even DSL can provide. Today, we tend to use NICs in our devices and rely on a MODEM-like

device that connects our home devices to our Internet service provider (ISP). This connection might be through fiber optic, coaxial cable, or twisted-wire pair directly to your home. Whatever method is used, the data transmitted are now binary data, allowing us to avoid modulation and demodulation.

1.2 TYPES OF NETWORKS

Now that we have some basic understanding of what a network is, let us differentiate between types of networks. We already mentioned client–server versus peer-to-peer networks. This is one classification of network types. Two other ways to classify networks is by their size and shape. The shape of a network is based on the topology. We explore that in [Section 1.2.1](#). The size of the network is not meant to convey the number of devices connected to it but the relative distance between the devices such as within a room, within a building, across a city, or larger. We explore the different types of networks by size in [Section 1.2.2](#). Keep in mind that the traditional form of a network is one where all the resources are in a relative close proximity, such as within one building or even a floor of a building. We will formally define a network whose components are in one local setting as a LAN.

Most networks on the planet are a form of LAN, and most of these use some form of wired connection; however, some devices might have a wireless connection to the network. Historically, devices within a LAN were often connected by either twisted-wire pair or coaxial cable, and some LANs still use these technologies. Today, it is more common to find LANs using fiber optic cable for connectivity.

1.2.1 NETWORK TOPOLOGY

The shape of a LAN is usually referred to as its *topology*. The topology dictates the way in which the devices of the network connect together. Early LANs (from the 1960s to the 1980s) may have connected along a single line, known as a bus. This formation yields a *bus* topology, also sometimes referred to as an Ethernet network, because Ethernet originally used the bus topology. The other early competitor is the *ring* topology, where each resource connects directly to its neighbor, thus a resource would connect to two other resources. [Figure 1.7](#) illustrates these two topologies, where the bus is shown on the left and the ring on the right.

The bus topology suffers from a significant problem. Because all resources share the single line, only one message can be transmitted over its length at any time. If multiple resources attempt to use the line at the same time, a collision occurs. Ethernet technology developed a strategy for handling collisions known as carrier-sense multiple access with collision detection (CSMA/CD). A device wishing to use the network will listen to the network first to determine if it is actively in use. If not, it will not only place its message onto the network but will also listen to the network. If this message collides with another, then the network will actually contain the two (or more) messages combined. Therefore, what is on the network now is not what the machine placed onto the network. If a collision is detected, the sending device sends out a jam signal to warn all other devices that a collision has occurred. Devices that caused the collision, along with other waiting

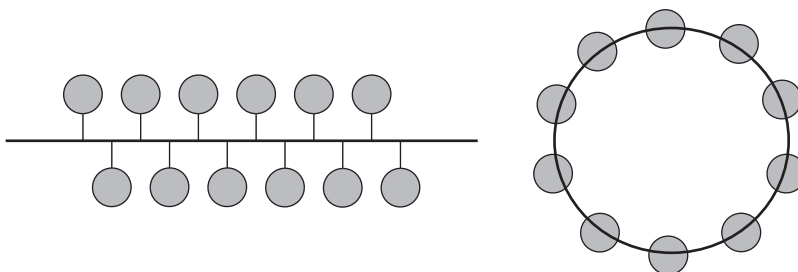


FIGURE 1.7 Bus and ring topologies.

devices, will wait a random amount of time before trying again. The bus topology is susceptible to traffic congestion when the network grows large enough so that resources may try to use the network at the same time.

The ring network does not suffer from congestion since messages do not share a single network but instead are sent from node to node until they reach their destination. However, the ring network will suffer in two ways. First, if any single node fails, it can isolate a portion of the network and keep some resources from communicating with others. Moreover, if the ring is large, communication latency increases, as any message will probably have a longer path to take to reach its destination. A ring can be unidirectional or bidirectional; the bidirectional ring helps mitigate these problems to some extent. For instance, consider a unidirectional ring of five resources, A, B, C, D, and E, where A sends to B, which sends to C, and so forth, while E sends to A. If E goes down, B, C, and D are unable to communicate with A at all. In a bidirectional ring, if E goes down, B, C, and D can still communicate with A by passing signals in the opposite direction around the ring.

A third topology, which is more expensive, is the *star* network. In such a topology, all resources connect to a single, central point of communication. This type of topology is far more efficient than the ring topology, because all messages reach their destination in just two (or fewer) *hops*. The star network is also more robust than the ring network because the failure of any node does not disconnect the network, unless it is the central node of course. However, the cost of the star topology is that you must now dedicate a resource to make up the central point of the network. This is typically a network device (e.g., hub and switch), which today is inexpensive and so worth the investment; however, it could also be a computer or even a server, which would be a more expensive piece of equipment. Another drawback of the star topology is that the central hub/switch will have a limited number of connections (e.g., 16), and so, the number of resources that make up the network will be limited. There are variants of a star network such as an extended star in which the two hubs or switches are connected. [Figure 1.8](#) illustrates some forms of star topologies. The upper left example is a standard star network. On the right, two star networks connect to a bus. In the bottom

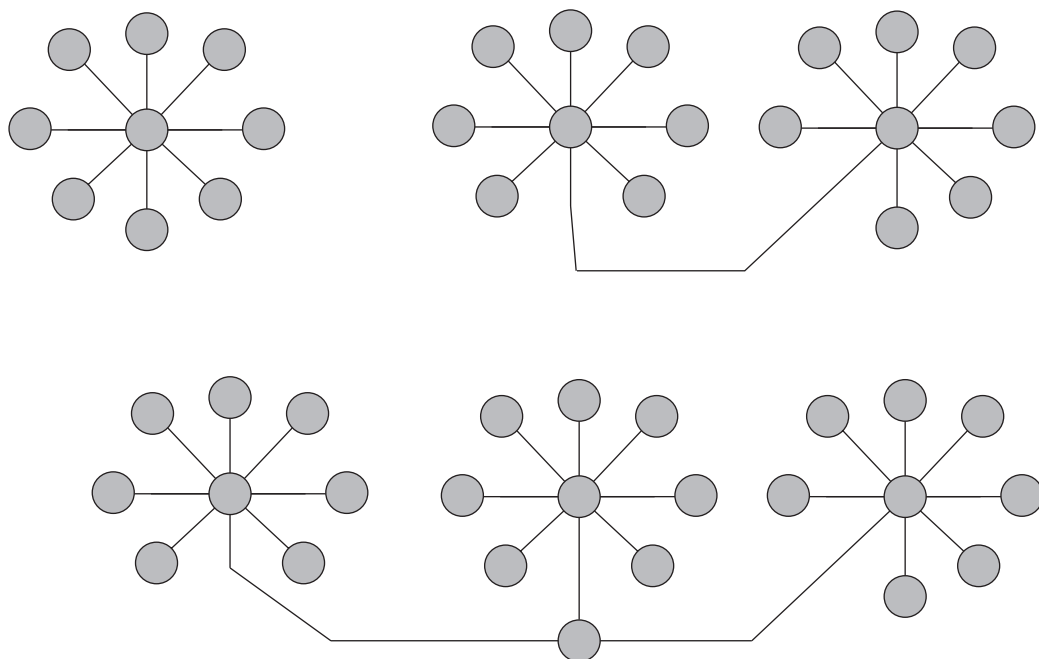


FIGURE 1.8 Various organizations of star topologies.

of the figure, three star networks are connected themselves to a star network, or there is a hierarchical star network, in which the three star network hubs connect to another hub.

A fourth form of network topology is the *mesh* or *nearest-neighbor* topology. Like the ring network, neighbors are directly connected. The ring network can be thought of as a one-dimensional (1D) nearest neighbor. Mesh networks can have larger dimensionality. For instance, a two-dimensional (2D) network would connect a resource to up to four neighbors (envision this as a mesh or matrix where a resource connects to the resources above, below, left, and right of it). A three-dimensional (3D) network adds another dimension for up to six connections. A four-dimensional (4D) network is often called a hypercube because it is harder to envision. At the most extreme, we have a fully connected network, where every resource has a direct connection to every other resource. This is prohibitively expensive and, in many cases, a waste of connectivity because there are some resources that might never communicate with other resources. Another variant of the nearest-neighbor topology is a tree topology, in which nodes at the top level are connected to some resources at the next level. Figure 1.9 demonstrates several of these nearest-neighbor layouts (a 1D and a 2D mesh on the upper left, a 3D mesh on the upper right, a hierarchical or tree mesh on the lower right, and a fully connected mesh on the lower left).

Today, a star network with a central switch or router is the most common form because it offers an excellent compromise between a reduction in message traffic, a robust network, and low latency of communication, all without incurring a great cost. Typically, the star network combines perhaps 10 to 20 resources (the upper limit is the number of connections available in the switch). Local area networks are then connected by connecting multiple switches to one or more routers.

Let us assume that we have a number of computer resources (computers, printers, file servers, and other types of servers) and a number of switches. The switches have 25 physical ports each, where one of these 25 ports is reserved to connect to a router. We connect 24 resources to the switch and then connect the switch to a router with the 25th connection. If our router also permits up to 24 connections, our local network will comprise 1 router, 24 switches, and up to 576 resources.

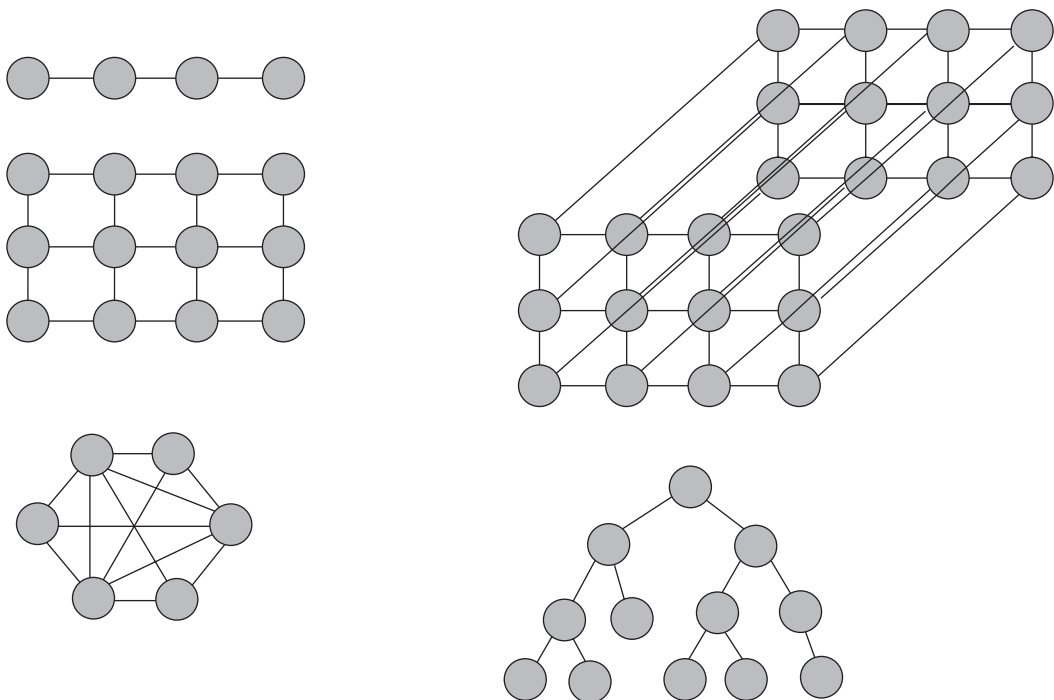


FIGURE 1.9 Nearest-neighbor topologies.

The router, like the switches, probably has one additional connection, so that we can connect the router to a gateway (our point of presence on the Internet).

1.2.2 CLASSIFICATIONS OF NETWORKS

The idea of connecting switches/routers together is known as *chaining*. Unfortunately, there is a practical limit to how many links can appear in a chain. For instance, you would not be able to expand the previous example to tens of thousands of resources. At some point, we will need repeaters to boost the signal coming from the most distant of resources, and eventually, we will also need routers.

Let us clarify the concept of chaining with an example. Consider a typical large organization's LAN. We will assume that the organization exists in one or several buildings, but it may be on several floors and with hundreds to thousands of resources to connect to the LAN. A floor might be hundreds of feet in length. We might find a few dozen resources on that floor. In one area of the floor, the local resources connect to a switch. Another area of the floor has another switch to support the resources near it. These switches (along with others) will connect to central router. This router might then connect to similar routers on other floors of the building. We would have similar connections in other nearby buildings. Buildings would be connected through additional routers. Our organization's LAN is no longer a single LAN but a LAN of LANs. Each floor of each building might be a single LAN. Alternatively, if there are enough resources on one floor, or if the building is particularly large, we might find multiple LANs on one floor. The organization's LAN is made up of smaller LANs. This could repeat, whereby smaller LANs are made up of even smaller LANs.

And so, we see that *LAN* is merely a term for a network that could possibly stretch for thousands of feet or be contained within one small area like a house. In order to better understand the size of LANs, multiple terms have been introduced. LAN often refers to a network of a few dozen to a few hundred resources in close proximity. A smaller network is the personal area network (PAN), which we will find in many households. The PAN might connect a few computers (including laptops with wireless cards) and a printer to a central router, which also connects the house's internal network to its ISP's network over fiber optic cable. At the other extreme, there is the campus area network (CAN), which is a network that stretches across many buildings of an organization. The term *campus* does not necessarily restrict this form to only universities, colleges, and schools but could also be applied to corporations that are held within a few buildings at one site. The most distant resources in a CAN might be as much as about a mile or two apart.

At the other extreme, we move from a LAN to a wide area network (WAN). Whereas a LAN will often be contained within, perhaps, a couple of square miles, the WAN is anything larger. There are a few classes of WANs. First, there is the metropolitan area network (MAN). A MAN might encompass a few miles or an entire metropolitan region. For instance, some cities have their own networks that provide resources such as event calendars, free access to information sources, free access to the Internet, and free email. The MAN is often connected by Ethernet-based cable but may also have other forms of connection, including wireless, in the form of radio signal or microwave. London, England, and Geneva, Switzerland, are two cities that have MANs. At a larger scope is a WAN that consists of specific sites within a country or across the world. These might include, for instance, offices of the US government and/or military, a university along with its branch campuses, or pay networks such as America On-line and CompuServe. The third form of WAN is the Internet itself. The distinction between a MAN, a WAN of specific sites, and the Internet is almost meaningless today because most long-distance connections are such that whether you are on a limited network or not, you still have access to the Internet. So, we might think of a MAN or a pay WAN as one in which you have full access to the Internet while also having access to certain resources that others do not have access to.

1.3 NETWORK PROTOCOLS

When you study networks, there are certain confusing terms that will arise again and again. These include request for comments (RFC), standard, protocol, protocol stack, and model. Although all these terms have related meanings, they are separate and important terms, so we need to discuss this before continuing. As stated previously, RFC stands for *request for comments*. An RFC is issued when a group (or an individual) working in the technology field wants to share some innovative approach to solving a problem within the field. They publish the RFC to solicit feedback. For instance, on implementing the Advanced Research Projects Agency Network (ARPANET), researchers posted RFCs for such networking needs as writing host software, defining the interfaces between computers and what was referred to as an interface message processor (IMP) and identifying different types of data messages. These were initially published in RFCs 1, 7, and 42, respectively, between 1969 and 1970. There are thousands of RFCs, and you can examine them at www.rfc-editor.org. Figure 1.10 shows the first portion of RFC 1591, which was the RFC regarding the structure and delegation of the DNS.

Once a group has obtained feedback through the RFC, the next step is to use the comments along with their own proposal(s) to define a set of standards that everyone in the field should adhere to. Given these standards, other researchers can begin to implement their own solutions. As long as an implementation follows the standards, the implementation should be usable by anyone else in the field following the same standards.

This leads us to the term *protocol*. Protocol, as defined in the English language, is a system of rules that explain proper conduct or communication. A *communication protocol*, or a *network protocol*, is a set of rules that determine how data is to be transmitted, addressed, routed and interpreted. The protocol meets the standards defined for the given problem. For instance, one protocol is the Domain Name System (DNS). The initial RFC for DNS was published in 1983 under RFC 882 and RFC 883. DNS, the protocol, was first implemented in 1984. Note that future RFCs for DNS have superseded RFC 882.

Most network protocols accomplish their varied tasks not through a single translation of a message into the network signals carried over the network but through a *series of* translations. This is

Network working group	J. Postel
Request for comments: 1591	ISI
Category: Informational	March 1994

Domain name system structure and delegation

Status of this memo

This memo provides information for the internet community. This memo does not specify an internet standard of any kind. Distribution of this memo is unlimited.

1. Introduction

This memo provides some information on the structure of the names in the Domain Name System (DNS), specifically the top-level domain names; and on the administration of domain. The Internet Assigned Numbers Authority (IANA) is the overall authority for the IP addresses, the domain names, and many other parameters, used in the internet. The day-to-day- responsibility for the assignment of IP addresses, autonomous system numbers, and most top and second level domain names are handled by the Internet Registry (IR) and regional registries.

2. The top level structure of the domain names

FIGURE 1.10 A portion of RFC 1591.

done by having layers in the protocol. When there are several layers, we typically refer to the series of mappings as a protocol *stack* because each layer is handled by a different protocol. A single layer in a protocol is a set of rules unto itself for translating the message into a new form, either to prepare it for transmission across the network or to prepare the transmitted message to be presented to the user.

One additional term is a *model*. When we speak of the Open Systems Interconnection (OSI) protocol stack, we are actually referring to a model because OSI has never been physically implemented. Some also refer to TCP/IP as a layered model because there are, in fact, many implementations for TCP/IP.

So, now that we have looked at types of networks, we can focus on what makes networks work. We have seen some of the hardware, but hardware is just a part of the story. In order for two resources to communicate, they must *speak* a common language. All communication over a network is eventually broken into bits, but devices on the network need to understand what those bits mean. We need a way to inform hardware and software when examining a message to be able to interpret such information as:

- Who the message is intended for (destination address)?
- What application software should utilize the message?
- How long is the message?
- Whether the message is complete or just one packet of many.
- Whether there is an error in the transmitted message and how to handle the error.
- How should a connection be established between the source and destination resources (if necessary)?
- Whether the message includes encoded data and/or is encrypted.

Such details are left up to the network protocol utilized to package up and transmit the message.

When a message is to be transmitted over network, it first starts as a product of some application. Therefore, the highest layer in a network protocol is typically the application layer. This message must be altered into a proper format. The application layer hands the message to another layer, which will add more detail to how the message is to be handled. For instance, it is common to break a message into smaller packets. A lower layer then must be responsible for breaking the message into packets. Each packet must be addressed and numbered in sequence so that the packets can be reassembled in their proper order. Another layer is responsible for these operations. The communication protocol then operates as a series of layers. Some layers may be implemented by a single protocol, but it is also likely that different protocols are applied at different levels.

In most cases, as a message moves down the protocol stack, information is added to the beginning of the message. This added content is known as a *header*, and each layer of the protocol stack might add its own header. In some cases, additional information can be added to the end of the message, in which case the added information is known as a *footer*. On receiving a message, the protocol stack operates in reverse, removing headers (and/or footers) as the message moves up the layers.

Numerous communication protocols are available. The most commonly used network protocol is TCP/IP, a protocol suite as mentioned previously. TCP/IP consists of four layers where each layer can be implemented through any number of possible individual protocols. However, TCP/IP is not the only network protocol suite. In [Sections 1.3.2 through 1.3.4](#), we examine some other protocols. We give a cursory look at TCP/IP in the first of these subsections. However, since this book is dedicated to the Internet, and TCP/IP is the protocol used throughout the Internet, we will spend a great deal more time examining TCP/IP through the text. Specifically, [Chapters 3 and 4](#) provide more detail on TCP/IP than what we cover here.

1.3.1 TRANSMISSION CONTROL PROTOCOL/INTERNET PROTOCOL

TCP/IP is composed of two separate protocol suites, TCP or the Transmission Control Protocol and IP or the Internet Protocol. Together, they comprise four layers. From top to bottom, we have the *application* layer, the *transport* layer, the *Internet* layer, and the *link* layer. TCP handles the top

two layers, whereas IP handles the bottom two layers. At each of the layers, there are numerous protocols that can be applied. That is, each layer has not one but many different implementations, depending on the needs behind the original message to be sent.

TCP/IP is utilized throughout the book. Rather than dedicating just one subsection to it, we defer most of the detail of this protocol to [Chapter 3](#) where we can dedicate the entire chapter to both the TCP/IP protocol stack and the individual protocols that can be used in each layer. We will make brief references to the four layers mentioned previously as we explore OSI. In [Chapter 4](#), we emphasize tools that a user or network administrator might use to inspect network connectivity and various aspects of TCP/IP.

However, before continuing, we briefly introduce two topics that described how we address devices in a TCP/IP network. All networks and devices are described through a network address. The most common form of address is an Internet Protocol version 4 (IPv4) address; however, we are also using IPv6 addresses across portions of the Internet. IPv4 addresses are 32-bit addresses and IPv6 are 128-bit addresses. In addition, messages sent from one device to another are sent using some type of protocol. The recipient device will use that protocol in order to interpret the sequence of bits in the message. This protocol is denoted by a port number. The port number indicates not just how to interpret the bits but also how to utilize the message. That is, the port dictates an application to utilize such as a web browser to use when the device receives a message encoded using the Hypertext Transfer Protocol (HTTP).

1.3.2 OPEN SYSTEMS INTERCONNECTION

Unlike TCP/IP, the OSI model is not a protocol suite because it is not implemented. Instead, it is referred to as a model with the intention of offering network architects a structure to target when developing new protocols. It was developed at roughly the same time as TCP/IP and has many overlapping or identical components. However, OSI does include a couple of features absent from TCP/IP.

OSI consists of seven layers. Most of the functionality of these seven layers can map to the four TCP/IP layers. OSI, like TCP/IP, dictates how a network communication is to be broken into individual units (packets) and how at each layer additional information is to be added to the message or packet as a header. The result is that the OSI packet, when transmitted, contains six headers (the bottom layer does not add a header), followed by the data that make up this portion of the message captured in this packet. In this subsection, we explore the seven layers and their roles. [Table 1.5](#) lists the seven layers and, for each layer, its role and the data unit used.

The seven layers are categorized into two layer types: the host layers and the media layers. The host layers provide details that support the data that make up the message. The media layers provide details used in the actual transmission and handling of each packet. As a message moves down the protocol stack, it is transformed, with header content prepended to it. These headers are stripped from the packet as the packet moves up the layers at the destination (or at an intermediate location).

TABLE 1.5
The Seven Layers of OSI

Layer Name	Layer Type	Layer Data Unit	Role
7. Application	Host layers	Data	Application software produces/displays message
6. Presentation			Application neutral format
5. Session			Maintain communication between hosts
4. Transport		Segments	Deal with errors and packet ordering
3. Network	Media layers	Packets/datagrams	Addressing and traffic control
2. Data link		Bit/frame	Connection between two network nodes
1. Physical		Bit	Network transmission

The top layer is the application layer. Here, some application software initializes the communication by generating the initial message. This message might be an HTTP request, as generated by a web browser to obtain a web page, or it might be some email server preparing to send an email message. On receipt of a message, it is at this layer that the message is delivered to the end user via the appropriate application software.

The application layer should be viewed not as a piece of software (e.g., Mozilla Firefox, Microsoft Outlook, and WinSock File Transfer Protocol [WS-FTP]) but instead as the protocol that the message will utilize, such as HTTP, FTP, or Simple Mail Transfer Protocol (SMTP). Other services that might be used at this layer are forms of resource sharing (e.g., a printer), directory services, authentication services such as Lightweight Directory Access Protocol (LDAP), virtual terminals as used by SSH or telnet, DNS requests, real-time streaming of data (e.g., RTSP), and secure socket transmission (Transport Layer Security [TLS]/Secure Sockets Layer [SSL]).

The presentation layer is responsible for translating an application-specific message into a neutral message. Character encoding, data compression, and encryption can be applied to manipulate the original message. One form of character encoding would be to convert a file stored in the Extended Binary Coded Decimal Interchange Code (EDCDIC) into the American Standard Code for Information Interchange (ASCII). Another form is to remove special characters like “\0” from a C/C++ program. Hierarchical data, as with Extensible Markup Language (XML), can be flattened here. It should be noted that TCP/IP does not have an equivalent function to this layer’s ability to encrypt and decrypt messages. Instead, with TCP/IP, you must utilize an encryption protocol on top of (or before creating) the packet, or you must create a tunnel in which an encrypted message can be passed. We explore tunneling in [Chapters 2 and 3](#). OSI attempts to resolve this problem by making encryption/decryption a part of this layer so that the application program does not have to handle the encryption/decryption itself.

The session layer’s responsibility is to establish and maintain a session between the source and destination resources. To establish a session, some form of *handshake* is required. The OSI model does not dictate the form of handshake. In TCP/IP, there is a three-way handshake at the TCP transport layer, involving a request, an acknowledgment, and an acknowledgment of the acknowledgment. The session, also called a connection, can be either in full duplex mode (in which case both resources can communicate with each other) or half-duplex mode (in which case communication is only in one direction).

Once the session has been established, it remains open until one of a few situations arises. First, the originating resource might terminate the session. Second, if the destination resource has opened a session and does not hear back from the source resource in some time, the connection is closed. The amount of time is based on a default value known as a *timeout*. This is usually the case with a server acting as a destination machine where the timeout value is server-wide (i.e., is a default value established for the server for all communication?). The third possibility is that the source has exceeded a pre-established number of allowable messages for the one session. This is also used by servers to prevent the possibility that the source device is attempting to monopolize the use of the server, as with a denial of service attack. A fourth possibility is that the server closes the connection because of some other violation in the communication. If a connection is lost abnormally (any of the cases listed above except the first), the session layer might respond by trying to reestablish the connection.

Some messages passed from one network resource to another may stop at the session layer (instead of moving all the way up to the application layer). This would include authentication and authorization types of operations. In addition, as noted previously, restoring a session would take place at this layer. There are numerous protocols that can implement the session layer. Among these are the Password Authentication Protocol (PAP), Point-to-Point Tunneling Protocol (PPTP), Remote Procedure Call Protocol (RPC), Session Control Protocol (SCP), and Socket Secure Protocol (SOCKS). The three layers described so far view the message itself. They treat the message as a distinct *data unit*.

The transport layer has two roles. The first role is to divide the original message (whose length is variable, depending on the application and the content of the message) into fixed-sized data

segments. Second, this layer is responsible for maintaining reliable transmission between the two end points through acknowledgment of the receipt of individual packets and through multiplexing (and demultiplexing) multiple packets carried in the same signal. If a packet fails to be received, this layer can request its retransmission. In essence, this layer is responsible for ensuring the accurate receipt of all packets.

OSI defines five classes of packets based on the needs of the application. These classes dictate whether the message requires a connection or can be connectionless, whether the packet is ensured to be error-free and/or utilizes any form of error handling, whether the packet can be multiplexed, whether retransmission of a dropped or missing packet is available, and whether explicit flow control is needed. TCP/IP forgoes these classes and instead offers two types of packets: TCP packets and User Datagram Protocol (UDP) packets.

The next layer is the network layer. At this layer, we finally begin to consider the needs of transmitting the packet over a network, that is, how this packet will be sent across the network. The data unit here is the *packet*, also referred to as a *datagram*. At this layer, network addressing is performed. In TCP/IP, the network address is an IP address (whether IPv4 or IPv6 or both), but the OSI model does not dictate any particular addressing method. Instead, there are several different protocols that can be utilized here for specific networks. These include, for instance, IPv4 and IPv6, and also Internet Control Message Protocol (ICMP) and Internetwork Packet Exchange (IPX), to name a few. The Address Resolution Protocol (ARP) maps network layer addresses to data link layer addresses and so bridges this network layer with the next layer down.

The network layer is required to support both connection-oriented and connectionless styles of transmission. Routers operate at this layer to route messages from one location to another. It is the router's responsibility to take an incoming packet, map it up to this layer to obtain the destination address, identify the best route to send the packet on its way to the destination, and forward the message on. Interestingly, since the message was decomposed into packets and the packets were transmitted individually, packets of the same message may find different routes from the source to destination devices. Although this layer handles forwarding a packet onto the next network location, it does not guarantee reliable delivery.

As discussed in [Section 1.1](#) of this chapter, switches operate on hardware addresses (we will see this in the next layer). However, there are layer 3 switches that operate at the network layer, making them synonymous with routers.

The second lowest layer of the OSI model is the data link layer. This layer is responsible for reliable delivery between two physical network nodes. The term *reliable* here means error-free from the point of view that the network itself is functioning, and so a packet placed at the source node will be received at the destination node. This level of error handling does not include errors that might have arisen during transfer, resulting in erroneous data (which would be taken care of at the transport layer). At layer 2, the content being moved is referred to as a *frame* (rather than a packet). Layer 2 protocols include asynchronous transfer mode (ATM), ARP (previously mentioned), X.25, and Point-to-Point Protocol (PPP). Both ATM and X.25, older protocols, are described in the online content that accompanies this chapter.

The data link layer comprises two sublayers, the MAC layer and the Logical Link Control (LLC) layer. The MAC sublayer maintains a list of all devices connected at this local network level, utilizing each device's hardware address, known as a MAC address. This is a 6-byte value usually displayed to the user in hexadecimal notation such as 01-E3-35-6F-9C-00. These addresses are assigned by the manufacturing and are unique values (although they can also be randomly generated by your operating system). The 6-byte value (48 bits) allows for 2^{48} different combination of addresses, which is a number greater than 280 trillion.

Frames that are intended for the given LAN (subnet) are delivered to a layer 2 device (a switch). The frame's destination MAC address is used to identify the specific device within this local network/subnet on which to send the frame. Unlike the router, which forwards packets onto another router (i.e., on to another network or subnetwork), the switch is, in essence, the end point. So, the

switch only needs to identify the layer 2 data from the frame, and therefore, the frame does not move further up the protocol stack at the switch.

The LLC sublayer is responsible for multiplexing messages and handling specific message requirements such as flow control and automatic repeat requests due to transmission/reception errors. Multiplexing is the idea of combining multiple messages into one transmission to share the same medium. This is useful in a traffic-heavy network as multiple messages could be sent without delay. It is the LLC sublayer's responsibility to combine messages to be transmitted and separate them on receipt.

The lowest layer of the OSI model is the physical layer. This is the layer of the actual medium over which the packets will be transmitted. Data at this layer consist of the individual bits being sent or received. The bits must be realized in a specific way, depending on the medium. For instance, voltage is placed on copper wire, whereas light pulses are sent over fiber optic cable. Devices can attach to the media via T-connectors for a bus network (which we will explore in [Chapter 2](#)) or via a hub.

We do not discuss this layer in any more detail, as it encompasses engineering and physics, which are beyond the scope of this text. As we examine other protocols both later in this chapter and in [Chapters 2 and 3](#), we will see that they all have a physical layer that is essentially the same, with the exception of perhaps the types of media available (e.g., Bluetooth uses radio frequencies instead of a physical medium to carry the signals).

On receipt of a packet, the OSI model describes how the packet is brought up layer by layer, until it reaches the appropriate level. Switches will only look at a packet up through layer 2, whereas routers will look at a packet up through layer 3. At the destination resource's end, retransmission requests and errors may rise to layer 4 and a loss of session will be handled at layer 5; otherwise, packets are recombined at layer 4 and ultimately delivered to some application at layer 7.

1.3.3 BLUETOOTH

No doubt, the name Bluetooth is familiar to you. This wireless technology, which originated in 1994, is a combination of hardware and protocol to permit voice and data communication between resources. Unlike standard network protocols, with Bluetooth one resource is the master device communicating with slave devices. Any single message can be sent to up to seven slave devices. As an example, a hands-free headset can send your spoken messages to your mobile phone or car, whereas a wireless mouse can transmit data to your computer. The Bluetooth communication is made via radio signals at around 2400 MHz but is restricted to a close proximity of just a few dozen feet. Bluetooth bandwidth is estimated at up to 3 megabits per second, depending on the transmission mode used.

The Bluetooth protocol stack is divided into two parts: a host controller stack dealing with timing issues and the host protocol stack dealing with the data. The protocol stack is implemented in software, whereas the controller stack is implemented in firmware on specific pieces of hardware such as a wireless mouse. These two stacks are shown in [Figure 1.11](#). The top stack is the host protocol stack, which comprises many different protocols. The most significant of these is the LLC and Adaptation Layer Protocol (L2CAP) (the "2" indicates two Ls). This layer receives packets from either the radio frequency communication (RFCOMM), the Telephony Control Protocol—Binary (TCS BIN), or Service Discovery Protocol (SDP) layer and then passes the packets on to either the Link Manager Protocol (LMP) or the host controller interface (HCI), if available. The L2CAP must be able to differentiate between these upper-layer protocols, as the controller stack does not. The higher-level protocols (e.g., wireless application environment [WAE] and AT-Commands) are device-dependent, whereas RFCOMM is a transport protocol dealing with radio frequencies. The controller protocol stack is largely hardware-based and not of particular interest with respect to the contents of this textbook.

Bluetooth packets have a default size of 672 bytes but can range from as little as 48 bytes to up to 64 kilobytes. Packet fields use Little Endian alignment (Little Endian is explained in Appendix B, located on the textbook's website Taylor & Francis Group/CRC Press). Because message sizes may be larger than the maximum packet size, the L2CAP must break larger messages into packets and reassemble and sequence individual packets on receipt. In order to maintain communication with

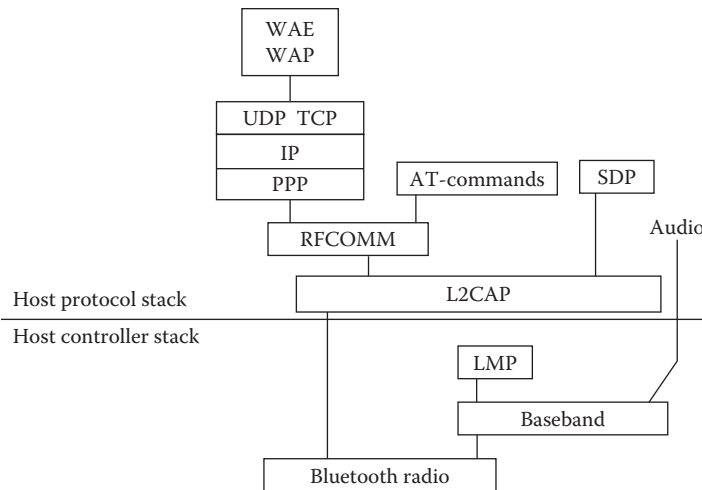


FIGURE 1.11 The Bluetooth protocol stack.

multiple devices, the L2CAP uses channel identifiers (CIDs) for each device with which it is communicating. Channel identifiers are uniquely assigned to each remote device. They can be reused if a device is no longer communicating with the host (however, some CIDs are reserved). Each channel is assigned one of three modes based on upper layers of the protocol. These modes are basic mode, flow-control mode, and retransmission mode.

The L2CAP layer also handles multiplexing of multiple data packets as well as reliability. Error handling is performed using the cyclic redundancy check (CRC) method (which we will explore in [Section 1.5](#)), dropped packets are handled by requesting retransmission, and timeouts result in packets being flushed from this layer.

1.3.4 FRAME RELAY

The original intention of the Frame Relay protocol was to support Integrated Services Digital Network (ISDN) infrastructures. Integrated Services Digital Networks provide communication for digital voice, video, and data combined, over the public switched telephone network (a circuit switched network, as opposed to packet switching of most computer networks). However, with the success of Frame Relay in ISDN, it was later extended to provide voice and data communication over packet-switched networks (we examine packet switching in [Section 1.5](#)), whether local area or wide area. In fact, the advantage of Frame Relay is to provide a low-cost means of transmitting digital voice/video/data from a LAN to an end point across a WAN. More specifically, a LAN connects to a Frame Relay network via a dedicated line. This line ends at a Frame Relay switch, which connects the LAN to other LANs.

Frame Relay communication breaks messages into variable-sized data units known as *frames*. A frame is typed by the type of data that it encapsulates (e.g., video, voice, and data). The end user is able to select a level of service that prioritizes the importance of a frame based on type. For instance, if voice data is deemed more significant, it will have a higher priority when being transmitted over the line connecting the LAN to the Frame Relay switch.

As it was felt that data transmitted using Frame Relay would be less error-prone than other forms of network connection, Frame Relay does not attempt to correct errors, thus reducing the amount of added data placed into a frame. An end point that receives an erroneous frame simply drops it and requests that it be resent. Video and voice data can often survive a few dropped or erroneous frames, unlike email or web page data. In a way, this distinction is similar to what we see between TCP and UDP (covered in [Chapter 3](#)).

Let us look at the frame as a piece of data to be transmitted. The standard Frame Relay frame consists of four distinct fields: a flag, an address, the data, and a frame check sequence (FCS). The flag is used for synchronization at the data link level of the network. All frames will start and end with the bit pattern of 01111110. In order to make sure that this bit sequence is unique in every frame, if this bit sequence were to appear in any other field, it would have to be altered by using an approach called bit stuffing and destuffing.

The address field is 16 bits. It consists of a 10-bit data link connection identifier (DLCI), an extended address, a control bit, and four congestion-control bits. The DLCI bits denote a virtual circuit so that this frame can be multiplexed with other frames of different messages. The DLCI is designed to be 10 bits in length but can be extended through the use of the extended address field to denote a longer DLCI. This might be the case if 10 bits is not enough to encode all of the virtual circuits (10 bits provide 1024 different identifiers). Next is the command/response (C/R) bit. This bit is currently unused. Finally, there are three bits dedicated to congestion-control information: the forward-explicit congestion notification (FECN), backward-explicit congestion notification (BECN), and discard eligibility (DE) bits. The FECN is set to 1 to indicate to an end device that congestion was detected en route. The BECN is set to 1 to indicate that the congestion was detected coming back from the destination to the source. The reason for these two bits is to help control the quality of delivery by avoiding or reducing the amount of data accumulation that might occur within the network itself. Finally, the DE bit denotes whether the given frame is of less importance and thus can be dropped if the network is determined to be congested.

The data field contains the transmitted data and is of variable length, up to 4096 bytes. The data in this field is placed here by a higher layer in the protocol (e.g., an application layer). Finally, the FCS contains a CRC value for the entire frame, as computed by the transmitting device. This provides an ability for error detection by the destination device. This field is 2 bytes in length.

An alternative to the standard frame format is known as the Local Management Interface (LMI) frame. We will not cover the details of the LMI frame, but it is better than the standard frame in a number of ways. For instance, an LMI frame provides global addresses and status information for synchronization of frames within a virtual circuit. It also provides a field for the type of frame.

Frame Relay had become very popular but is becoming less and less supported by ISPs because of cheaper technologies such as fiber optic cable reaching more and more end points. In fact, Frame Relay as a topic has been removed from the popular Cisco Certified Network Administrator (CCNA) examination.

See the textbook's website at Taylor & Francis/CRC Press for additional readings on several older network protocols.

1.4 ETHERNET

Ethernet is a combination of hardware and low-level protocol. We separate this from [Section 1.3](#), because Ethernet is primarily concerned with hardware. Ethernet began development at Xerox PARC in the 1970s but was released by 3Com in 1980. Ethernet is a family of technologies used for LAN design and implementation. Although its origins date back to 1980, it is still the most commonly used form of LAN. With time, the Ethernet model has been enhanced to utilize newer technologies so that it can be improved regarding such aspects as bandwidth. Today, we even find Ethernet being used in MANs.

Ethernet is largely concerned with just the lowest layers of the network, the physical layer and the data link layer. At the physical layer, Ethernet specifies a number of different variations of wiring and signaling. These include the use of all three common forms of cabling: twisted pair, coaxial cable, and fiber optic cable. The original Ethernet cabling was twisted-pair cabling, keeping Ethernet networks cheaper than many competitors, but it had a limited upper bandwidth of

3 megabits per second. Later, with improvements made to these forms of cable, newer names have been provided using the cable's bandwidth rating: 10BASE-T, 100BASE-T, and 1000BASE-T (10 for megabits per second, 100 for 100 megabits per second, and 1000 for 1000 megabits or a gigabit per second, respectively). Today, 10GBASE-T and 100GBASE-T are also available.

In addition to the cable and computer resources, an Ethernet network requires repeaters to handle signal degradation. We used the term *repeater* earlier in this chapter when we introduced a hub. A hub's job is to pass a message on to other resources. Thus, it repeats the message to all components connected to it. In an Ethernet network, the repetition was required to boost the signal so that it could continue to travel across the network. The maximum length of cable that could be used without a repeater was thought to be around 300 feet (roughly 100 meters). The repeater, also known as an Ethernet extender, would allow an Ethernet network to encompass a larger size. Today, the maximum distance between Ethernet repeaters is thought to be about 6000 feet. Early repeaters had just two ports so that the repeaters made up a bus topology network. However, with the advent of true hubs used as repeaters, Ethernet networks could take on a star topology. Refer to [Figure 1.2](#), which illustrates this difference.

To connect to the network, resources required an adapter. There were specialized adapters created for many mainframe and minicomputers of the era. In 1982, an adapter card was made available for the IBM PC.

Because of early Ethernet networks being a bus topology, there was the risk of multiple messages being placed on the network at the same time. In order to handle this, Ethernet introduced CSMA/CD, which operates at the data link layer. Any resource that attempts to place a message on the network will first listen to the network for traffic. If there is traffic, the resource will wait and try again at the next *time slot*, until the network becomes available. However, what if other resources are also waiting? When the network becomes available, if all of them attempt to use the now-free network, their messages will collide. Therefore, each resource not only listens to ensure a free network but also listens once the resource's message has been placed on the network for the possibility of a collision. On detecting a collision, the resource sends out a special message called a jam signal. This signal informs all resources of a collision. Any resource that was trying to use the network now knows of the collision. Each resource then waits for a random amount of time before trying again. Since the random amount should differ device by device, a resource's second attempt should not take place at the same time as another resource that was involved with the same collision. Again, the resource listens to the network first, and then, if or when free, it tries again.

Note that although repeaters help extend the physical size of a network, all devices in a bus network are susceptible to collisions. Thus, the more resources added to a network, the greater the chance of a collision. This limits the number of resources that can practically be connected to a LAN. Moving from a bus topology to a star topology, where the central point is a switch, can alleviate this problem. In 1989, the first Ethernet switch was made available for Ethernet networks.

To wrap up Ethernet, we now briefly focus on the data link layer. In this layer, Ethernet divides a network message into packets, also known as frames. The description of a packet is shown in [Figure 1.12](#). Here, we see that the packet contains a 56-bit preamble. This section consists solely of alternating 1 and 0 bits, as in 10101010, repeated for 7 total bytes. The preamble is used to synchronize the packet coming in from the network with the computer's clock. This is followed by a 1-byte start frame delimiter consisting of the binary number 171 (10101011). Notice how this sequence is nearly identical to a preamble byte, except that the last bit is a 1. This informs the receiving computer that the packet's information is about to begin.

Next in the packet is the packet header. The header contains two types of information, addresses and packet type. The addresses are MAC addresses for both the destination computer and the source computer. These fields are 6 bytes apiece. Optionally, an 802.1Q tag of 2 bytes is allowed. This is strictly used if the network is a virtual LAN or contains virtual LAN components. Otherwise, this field is omitted. The last portion of the header is a 2-byte field that specifies the type of frame, the size of the data portion (the *payload*), and, for large payloads, the protocol being used. The types of frames are Ethernet II frames, Novell frames, LLC frames and Subnetwork Access Protocol frames.

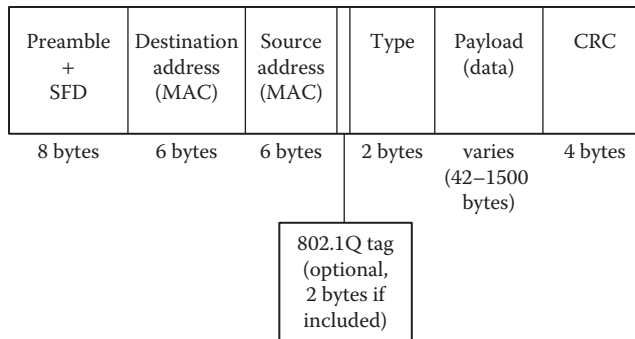


FIGURE 1.12 Ethernet packet format.

The payload follows, which is at least 42 bytes in size but can be as large as 1500 bytes. However, special packets known as jumbo frames can have as many as 9000 bytes. Following the data section is an error-detection section known as the FCS. This is a 4-byte segment that contains a CRC value, as computed by the transmitting device. The receiving device will use the CRC value to ensure the integrity of the data. On detecting an error, the receiving device drops the packet and requests a retransmission. The packet is now complete, and what follows will be an interpacket gap. However, in some cases, the physical layer can add its own end-of-frame sequence.

Ethernet is significant both historically and in today's networks. We will return to Ethernet in [Chapter 2](#) and add detail to this introduction.

1.5 THE INTERNET: AN INTRODUCTION

In this section, we take an introductory look at the Internet. We focus on what makes the Internet work, at a shallow level. We explore the Internet backbone, Internet routers, TCP/IP, and the DNS. As [Chapter 3](#) covers TCP/IP in detail and [Chapter 5](#) covers DNS in detail, our look here is void of most of the details that you will see in later chapters.

The Internet is a physical network. By itself, it does not provide services. Instead, the services are made available through servers that run applications and the protocols developed so that the services can be made available, no matter what type of computer or network you are using to communicate with those servers. The Internet is the mechanism by which the communications are carried from one location to another, from the requesting computer to the server, and back to the requesting computer. Thus, we need to understand that the Internet was developed independent of the services that we use over the Internet.

See the textbook's website at Taylor & Francis/CRC Press for additional reading about the history of networking and the history of the Internet.

1.5.1 WHAT IS A NETWORK ADDRESS?

As mentioned earlier, a network protocol will proscribe a means of addressing. The OSI model dictates two layers of addresses: layer 2 addresses and layer 3 addresses. The reason for this distinction is that layer 2 addresses will be recognizable only within a given network based on a stored table by the network's switch. The switch obtains the address whenever a new device is connected to that switch's local network. As opposed to that, routers need to use a different form of address. So, we differentiate between layer 2 addresses, which are usually called hardware addresses, and layer 3 addresses, which we will call network addresses.

Hardware addresses are also known as MAC addresses, and they are assigned to a given interface at the time the interface is manufactured (however, most modern devices' MAC addresses can be adjusted by the operating system). Instead, layer 3 addresses are assigned by devices when they are connected to the network. These can be statically assigned or dynamically assigned. If dynamically assigned, the address remains with that device while the device is on the network. If the device is removed from the network or otherwise shut down, the address can be returned to the network to be handed out to another device.

The most famous protocol, TCP/IP, utilizes two types of addresses, IPv4 and IPv6. An IPv4 address consists of 32 bits divided into four *octets*. Each octet is an 8-bit number or a decimal value from 0 to 255. Although IPv4 addresses are stored in binary in our computers, we view them as four decimal values, with periods separating each octet, as in 1.2.3.4 or 10.11.51.201. This notation is often called *dotted-decimal notation* (DDN).

With 32 bits to store an address, there are a possibility of 2^{32} (4,294,967,296) different combinations. However, not all combinations are used (we will discuss this in [Chapter 3](#)), but even if we used all possible combinations, the 4 billion unique addresses are still not enough for today's Internet demands. In part, this is because we have seen a proliferation of devices being added to the Internet, aside from personal computers and mainframes: supercomputers, servers, routers, mobile devices, smart TVs, and game consoles. Since every one of these devices requires a unique IP addresses, we have reached a point where IP addresses have been *exhausted*. One way to somewhat sidestep the problem of IP address exhaustion is through network address translation (NAT), which we will discuss in [Chapter 3](#). However, moving forward, the better strategy is through IPv6. The IPv6 addresses are 128-bit long and provide for enough addresses for 2^{128} devices. 2^{128} is a very large number (write 3 followed by 38 zeroes and you will be getting close to 2^{128}). It is anticipated that there will be enough IPv6 addresses to handle our needs for decades or centuries (or more). One similarity between IPv4 and IPv6 addresses is that both forms of addresses are divided into two parts: a network address and an address on the network (you might think of this as a zip code to indicate the city and location within the city, and a street address to indicate the location within the local region). We hold off on specific detail of IPv4 and IPv6 addresses until [Chapter 3](#).

It is the router that operates on the IP addresses. It receives a packet, examines the packet's header for the packet's destination addresses, and decides how to forward the packet on to another network location. Each forwarding is an attempt to send the packet closer to the destination. This is known as a packet-switched network to differentiate it from the circuit-switched network of the public telephone system. In a circuit-switched network a pathway is pre-established before the connection is made, and that pathway is retained until the connection terminates. However, in a packet-switched network, packets are sent across the network based on decisions of the routers they reach. In this way, a message's packets may find different paths from the source to the destination. En route, some packets may be lost (dropped) and therefore require retransmission. Although this seems like a detriment, the flexibility of packet switching allows a network to continue to operate even if nodes (routers or other resources) become unavailable. As IPv6 is a newer protocol, not all routers are capable of forwarding IPv6 packets, but any newer router is capable of doing this.

Aside from MAC addresses and IP addresses, there are other addressing schemes that are or have been in use. With Bluetooth, only specialized hardware devices can communicate with this protocol. These devices, such as NICs, are provided with a hardware address on being manufactured. This is known as the BD_ADDR. A master device will obtain the BD_ADDR at the time the devices first communicate and use the BD_ADDR during future communication.

X.25 (described in the online readings) used an 8-digit address comprising a 3-digit country code and a 1-digit network number, followed by a terminal number of up to 10 digits. The OSI model proscribes two different addresses: layer 2 (hardware addresses) and layer 3 (network service access point [NSAP] addresses). However, the OSI model does not proscribe any specific implementation for these addresses, and so, in practice, we see MAC and IP addresses, respectively, being used. Ethernet and Token Ring, as they only implement up through layer 2, utilize only hardware addressing.

One last form of addressing to note is that used in the Internetwork Packet Exchange (IPX). It is a part of the IPX/SPX protocol suite used in Novell NetWare networks and is found on many Microsoft Windows-based networks of machines that use a Windows OS (up through Windows 95). An IPX address consists of three fields: a 4-byte network address, followed by a 6-byte node number, followed by a 2-byte socket number. These are synonymous to the IP address network address, machine address, and port address, respectively (we will discuss ports in [Chapter 3](#)). However, unlike the IP address, where a portion of the entire address is dedicated to the network and a portion is dedicated to the device address, here, the two parts are kept distinctly separate. Sockets were assigned as part of the protocol so that, for instance, a socket number of 9091 (in hexadecimal) carried TCP packets, 9092 carried UDP packets, and 0455 carried NetBIOS data.

1.5.2 ERROR HANDLING

There are many forms of error handling in computers. Some forms exist within the computer to handle simple data-transfer problems, such as parity bits. Other forms are used to provide data redundancy in storage (RAID devices). For telecommunications, we need a mechanism that is not time-consuming to compute but provides a high accuracy of detecting an error. As noted in [Section 1.3](#), many of the forms of protocols can utilize error detection so that any packet that contains an error can be retransmitted. This is critical if the packet contains data that must be correct, such as an email message or an HTTP request.

There are many reasons why a transmitted packet might arrive with erroneous data. The most common reasons revolve around the nature of the medium being used to transmit the data and the length of that medium. For instance, copper wire carries electromagnetic data. This can be influenced (interfered with) by a number of different external bodies, including atmospheric conditions (e.g., lightning), solar flares, and a strong electromagnetic field in the vicinity. Copper wire also has a very limited length under which an electronic signal can be carried without using some form of repeater. Wireless communication can be interfered with by other radio signals at similar frequencies as well as by atmospheric conditions. However, even fiber optic cable is not fault-free. Errors can always arise when transmitting data, especially when the transmission rate is as high as billions of bits per second.

Therefore, network communication requires a means of error detection. This is handled by computing some form of error-detection data. This data is then attached to the message somewhere (usually as an extra field in the message's header). The transmission of the message consists of two parts: the message and the error-correction information. Typically, the mechanism is set up only to detect an error. Error-correcting codes are another concept and can be used for telecommunications, but in practice, they are not used for telecommunication because they take more time to compute and require more bits, thus making a message longer.

Error detection is handled in OSI layer 4. At this layer, error-detection information is computed by the transmitting computer and is added to the message. The message is then transmitted. The recipient breaks the message into its parts, identifying the error-detection data. It then performs a check on the message by determining if the message and error-detection data match. If the recipient does not find a match, it knows that an error occurred somewhere in the course of the traversal of the message over the network and asks for the message to be resent. If the error-detection data match with the message, the assumption is that no error arose. This may be a faulty assumption, as we will see when we look at checksums later, but it is typically the case that an error will lead to incorrect error-detection data. See [Figure 1.13](#).

The most common forms of error-detection algorithms used in data communication are checksums and CRC. Although both are the same idea, they are implemented in different ways. We'll discuss both, starting with the simpler one, the checksum.

Assume that a message consists of n bytes and we want our checksum to be stored in 1 byte. Since any byte of binary data can represent a positive integer decimal number (just convert from binary to decimal), we take the n bytes and add them together. We divide by the value 2^8 (256) and

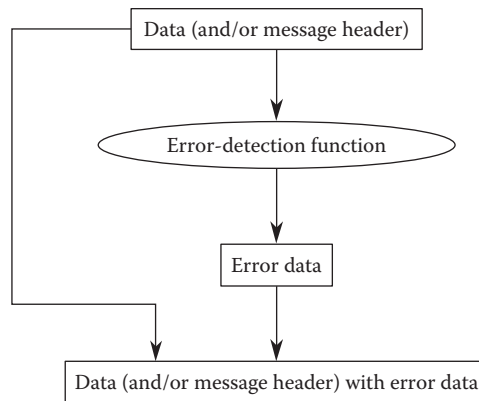


FIGURE 1.13 Adding error-detection data to a message.

take the remainder. The remainder of a division is computed using the modulo operator. Computing the remainder is actually done for us whenever a division is performed (computer hardware computes both the quotient and the remainder at the same time).

Let us consider as an example that our message is the word “Hello” (without the quote marks). This string of five characters is stored in ASCII by using 1 byte per character (ASCII actually represents characters in 7 bits, with the 8th bit often unused). Thus, our message is

01001000 01100101 01101100 01101100 01101111

These five bytes in decimal are 72, 101, 108, 108, and 111, respectively, and their sum is 500, or the binary number 111110100. Notice that our sum is stored in 9 bits instead of 8 bits. Now, we divide this by 256 and take the remainder. The remainder, when divided by 256, will always be between 0 and 255. Thus, we are guaranteed that the resulting checksum can be stored in 1 byte. In this case, $500/256$ is 1, with a remainder of 244. Our checksum is 244, or 11110100. Notice that this number is the same as the rightmost 8 bits of sum.

Notice how our sum and our checksum are related in that our checksum is actually just the rightmost 8 bits of our sum. Consider two other numbers, which, when divided by 256, result in the same remainder: 756 (which is 1011110100) and 1012 (which is 1111110100). These two numbers as well as 500 provide a remainder of 244 (11110100). In fact, 756 is $500 + 256$, which we obtain by adding a 10 to the left of the binary number for 244, and 1012 is $500 + 512$, which we get by adding bits of 11 to the left of the binary number for 244. In all three cases (500, 756, and 1012), the rightmost 8 bits are the same. We can surmise from this that with an 8-bit checksum, numbers that are exactly 256 apart from each other will have the same checksum. Given any two messages, if their sum is exactly 256 apart, then they have the same checksum.

Why should we care that two possible messages have sums that are 256 apart from each other? Imagine that the message transmitted above were not as shown but instead

00001000 00100101 00101100 00101100 01101111

Here, the sum is 244, and so, the checksum is 244. If we transmit “Hello” and receive the above message, it still looks correct based on the checksum. It is a very different message, because the leading 1 in the first four bytes was dropped, resulting in the ASCII message “\b ,o” (a backspace, a unit separator, two commas, and the only surviving character, the “o”). For this reason, we might want to use a larger checksum, for instance, 16 bits. A 16-bit checksum requires adding 16-bit values together and then dividing by 65,536 (2^{16}). For our ASCII example, we might then add 0100100001100101 (the ASCII values for “h” and “e”) to 0110110001101100 (the ASCII values for “l” and “l”) and 0110111100000000 (the ASCII value of “o” followed by an empty byte).

As division is a time-consuming process, another way to perform a checksum computation is to sum the bytes (or 16-bits, if we are using a 16-bit checksum) together. We then take the carry

produced by the summation and add that carry to the sum. Finally, we take the one's complement of this sum. To compute a one's complement number, we flip all the bits (every 1 becomes a 0 and every 0 becomes a 1). Let us examine this with our same message "Hello" using an 8-bit checksum. As we already saw, the summation of the 5 bytes in "Hello" is 11110100. As an 8-bit number, we have 11110100 with a carry of 1 (the leading bit is a 1). Summing the carry and the sum gives us $11110100 + 1 = 11110101$. The one's complement of this is 00001010. Our checksum is 00001010 or the decimal number 10 (not to be confused with a binary number).

We can test our checksum by summing the original message and the checksum. We then add the carry to our sum. Finally, we take the one's complement value, and the result should be 0. Let us see if this works. We already know that the sum of the 5 bytes is 11110100; to this, we add 00001010 to give us 11111110. The carry here is again the leading 1 bit. We add the first 8 bits of our sum and our carry, and we get $11111110 + 1 = 11111111$. Taking the one's complement of this gives us 00000000. Therefore, if we receive the 5 bytes and the checksum correctly, we see that there is no error.

Although the checksum is an easy-to-compute and a robust means of detecting errors, the CRC method is the preferred means of handling network communication error detection. There are several reasons for this. The primary reason is that the checksum is the older and less sophisticated approach and so is more susceptible to problems of not detecting errors if they exist in the message. In addition, the checksum is useful when detecting single-bit errors but less useful when a message may contain multiple-bit errors (i.e., multiple bits were altered).

Cyclic redundancy check uses a similar idea as the checksum in that it produces an n-bit result to be tacked onto a message. However, the computation differs. Instead of summation and division, we repeatedly perform an exclusive OR (XOR) operation on the bits of the message, along with a designated key value.

Before we begin this examination, let us get an understanding of the XOR operation. XOR is a bit-wise operation performed on two bits. The result of the XOR is 1 if the two bits differ (one bit is a 1 and the other bit is a 0) and 0 if the two bits are the same (both bits are 1 or both bits are 0). As XOR is a simple operation for computer hardware to perform, a series of XOR operations can be done very rapidly.

For CRC to work, we have to first define the *key*. The key is an n-bit value, where the value must be agreed upon by both the transmitting and receiving parties. We will assume that the key starts with and ends with 1 bits, because such a key is easier to utilize and provides for better results. Note that although the transmitting and receiving parties do not have to agree upon a key, the key is incorporated into the protocol, as will be described later. The algorithm works as follows, assuming a key of n bits.

1. Pad the message with n-1 0 bits at the end (on the right) and remove all leading 0s from the message, so that the message's first bit is a 1.
2. Align the key under the message at the leftmost end (both key and message start with 1 bits, since we removed all leading 0s).
3. Perform a XOR between the n bits of that portion of the message and the n-bit key. Note that the result of the XOR will always start with a 0, because the first bit of the key and the first bit of the message are both 1s (1 XOR 1 is 0).
4. Realign the key with the first 1 bit from the result in step 3. Repeat steps 3 and 4 until the leading 1 bit of the message has fewer bits than the key.

The error-detection data are the remaining bits.

Let us look at an example. We will again use the message "Hello" along with an 8-bit key whose value is 10000111. For step 1, we have to slightly modify the message by dropping the leading 0 and adding 7 zeroes to the end. Thus, our message changes from

01001000 01100101 01101100 01101100 01101111

to

1001000011001010110110001101100011011110000000.

Now, we align the key to the message, as stated in step 2, and apply steps 3 through 4 repeatedly, until we reach the terminating condition (the result of the XOR has fewer bits than the key).

To continue with this example, we use the notation M for the message, K for the key, and X for the result of the XOR operation between M and K. K is aligned under the message. We compute X and then *carry down* the remainder of M to form a new M. Notice that after the 5th iteration, there should be a leading zero carried down to M (the 0 is indicated with an underline under the 5th iteration's value for M), but we do not carry down any leading zeroes, so we start with the next sequence of bits after the zero (11011000).

```

M      1001000011001010110110001101100011011110000000
K      10000111
X      00010111
M      1011111001010110110001101100011011110000000
K      10000111
X      00111001
M      11100101010110110001101100011011110000000
K      10000111
X      01100010
M      1100010010110110001101100011011110000000
K      10000111
X      01000011
M      100001110110110001101100011011110000000
K      10000111
X      00000000
M      110110001101100011011110000000
K      10000111
X      01011000
M      10110001101100011011110000000
K      10000111
X      00110110
M      110110101100011011110000000
K      10000111
X      01011101
M      10111011100011011110000000
K      10000111
X      00111100
M      111100100011011110000000
K      10000111
X      01110101
M      11101010011011110000000
K      10000111
X      01101101
M      1101101011011110000000
K      10000111
X      01011010
M      101101011011110000000
K      10000111
X      00110010
M      1100101011110000000
K      10000111
X      01001101
M      100110111110000000
K      1000111
X      0001010
M      110111110000000

```

K	1000111
X	0101000
M	10100010000000
K	10000111
X	00100101
M	100101000000
K	1000111
X	0001100
M	110000000
K	10000111
X	01000111
M	10001110
K	10000111
X	00001001

Our last XOR result, 00001001, is less than our key, 10000111, so we are done. The message, when the 8-bit CRC key is applied, gives us the value 00001001. We will call this value C. We affix C to the message before transmitting it.

At the receiving end, based on the protocol used, the recipient knows to remove the CRC value, C or 00001001, from the message. The recipient computes $M + C$ to get M_2 . Now, the recipient goes through the same process as above using M_2 and K instead of M and K. The result should be the remainder 00000000. If so, no error occurred. If the remainder is anything other than 00000000, an error arose.

In practice, many forms of telecommunication use either 16-bit or 32-bit CRC known as CRC-16 and CRC-32, respectively. That is, they use CRC with either 16-bit or 32-bit key lengths. Both X.25 and Bluetooth use a version of CRC-16 known as CRC-16-CCITT, with a key defined as 1000000100001. Ethernet uses CRC-32, with a key defined as 10011000001000111011011011. TCP/IP, sending either a TCP or UDP packet, uses a 16-bit checksum.

1.5.3 ENCRYPTION TECHNOLOGIES

We now take a brief look at encryption and encryption technology. The primary reason that we require encryption is that messages sent on the Internet (and over most networks) by default are in ordinary text. That is, the bits that make up the data portion of a message are just ASCII (or Unicode) bits. It is possible, through a number of different mechanisms, that a message could be intercepted by a third party between its source and destination. For instance, a router could be programmed to forward a message both to the next logical hop along the network and to a specific IP address. Alternatively, a network router could be reprogrammed so that the IP addresses that it stores are altered. Known as ARP poisoning (among other terms), it could permit someone to take control of where that router forwards its messages to. Yet another possibility is that a wireless network could be *tapped into* if someone is nearby and can see the message traffic going from someone's computer to his or her access point.

The threat that messages can be intercepted over the Internet may not be a particular concern if it weren't for using the Internet to access confidential information. If insecure, someone might be able to obtain the password to one of your accounts and, with that password, log in to your account to transfer money, spend money, or otherwise do something malicious to you. Alternatively, an insecure message might include a credit card or social security number, which could allow a person to steal your identity.

In order to solve the problem of an Internet that is not secure, we turn to *data encryption*. Encryption is the process of translating a message into a coded form and thus making it difficult for a third party to read the message. Decryption is the opposite process of taking the coded message

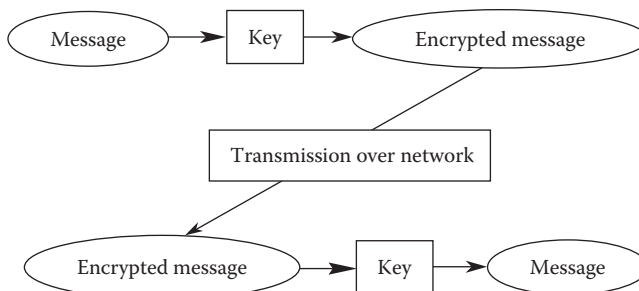


FIGURE 1.14 Encrypting and decrypting a message.

and translating it back to its original form (Figure 1.14). As we will see in the following discussion, we might have one key to use for both encryption and decryption or two separate keys.

Encryption and decryption have been around for as long as people have needed to share secret information in communicated forms. In wars of the twentieth century, encryption and decryption were performed using replacement codes. In such a code, each letter would be replaced by another letter. For instance, common replacement codes are known as Caesar codes, in which an integer number is used to indicate the distance by which a character should be changed to. For instance, a code with a distance of 1 would have “hello” converted into “ifmmp.” This is also known as a rotation + 1 code. A slightly more complicated code is a Vigenere cipher, in which several Caesar codes are applied, where the distance for each given letter is indicated by a special keyword. Without knowing the replacement technique used (whether rotation, keyword, or some more complex replacement algorithm), someone can still break the code by trying all possible combinations of letters. This is known as a brute-force approach to decryption. Unfortunately, replacement codes are far too easy to break by computer, which can try millions or more combinations in a second.

However, today, we use a more sophisticated type of encryption technology. Consider that our message is converted to be a binary message (because we are transmitting information by computer). We take the message and break it into units of n bits each. We use n integer numbers. For each 1 bit in the message, we add that corresponding integer number. For instance, if we have $n = 8$ and our numbers are 1, 2, 5, 10, 21, 44, 90, and 182, then the message 10110010 would be encrypted as the sum of 1, 5, 10, and 90 or 106, and if our message was 00011101, our message would be encrypted as the sum of 10, 21, 44, and 182 or 257. We will call our numbers (1, 2, 5, 10, 21, etc.) our *key*.

So far so good. On receiving the message as a sequence of integer numbers one per every 8 bits of the message, we restore the original values. How? Notice that our key of numbers is *increasingly additive*. That is, each number is greater than the sum of all the preceding numbers put together. 5 is greater than $1 + 2$. 10 is greater than $1 + 2 + 5$. 21 is greater than $1 + 2 + 5 + 10$. This allows us to identify the numbers that make up our received number (say 257) by finding the largest number in our key that can be subtracted from the received number. In 257, we know that there must be a 182, since 182 is the largest number less than or equal to 257, and all the numbers less than 182, if added together, will sum to less than 182. That is, if we do not use 182 as part of our message, we cannot come close to 257 because $1 + 2 + 5 + 10 + 21 + 44 + 90 < 182$. So, we subtract 182 from 257, giving us 75. The next largest number in our key ≤ 75 is 44, so now we know that 257 includes both 182 and 44, leaving us with 31. The next number must then be 21, leaving us with 10. The next number must be 10. Therefore, $257 = 10 + 21 + 44 + 182$, or the 4th, 5th, 6th, and 8th numbers in the key, which give us 257, and so the 8-bit value transmitted must be 00011101. We have successfully restored the original message!

This approach that we just described is known as *private-key* encryption or *symmetric-key* encryption. The key is known only to one person, and that person uses the key to both encrypt and decrypt data. This is fine if we want to encrypt our files on our computer for secure storage, because, by knowing the key, we can decrypt the files any time we want. Or, if we share the key with someone, we can then encrypt the message, send it over the Internet, and that person can use the same key to decrypt it. However, this approach cannot work for e-commerce. Why is this? The company that runs the website (say amazon.com) needs to be able to decrypt your messages, so that it can get your credit card number. However, you also need to have the key so that you can encrypt the messages to begin with. If the company shares the key with you, what is there to prevent you from intercepting someone else's messages and using the key to decrypt their communications with amazon.com?

With a private key, we can also generate another key called a *public* key. The public key does not have the increasingly additive property that we saw with the list above. For instance, it is possible that a key of 8 could have the values 31, 6, 22, 58, 109, 4, 77, and 21. If amazon.com provides you with a public key to encrypt a message, you cannot use it to easily decrypt a message. Of course, if you use this key, the message 00011101 will not be encrypted to 257 but instead 192. If amazon.com uses its private key to decrypt 192, it will not get 00011101. To solve this problem, there are extra numbers generated along with the public key, whereby these numbers are used to convert 192 into 257. Amazon.com then applies its private key to decrypt the message.

This form of encryption is called *public-key* encryption (also *asymmetric-key* encryption). Here are the concepts in summary.

- Someone creates a private key that he or she keeps secret.
- From the private key, he or she can generate a public key and share the public key.
- A message is converted from its original binary form into a sequence of numbers by applying the public key.
- These numbers are transmitted *in the open* across the network.
- On receiving the numbers, they are converted into different numbers.
- Those new numbers are used with the private key to decrypt the message.

In order to support public-key encryption, we have the public key infrastructure (PKI). The PKI comprises both the technology and policy guidelines to create and manage public keys and their dissemination. This includes, for instance, the ability to generate digital certificates (which embed public keys within them), certify such certificates as authentic, and alert users when a certificate is being revoked. We will discuss the process of creating, authenticating, issuing, and revoking certificates in [Chapter 3](#) and utilizing certificates in the Apache web server in [Chapter 8](#).

You might wonder why someone could not use the public key to decrypt the original message. For instance, if you know that the message is 192, couldn't you identify the numbers from the public key that add up to 192? The answer is yes, but it cannot be done as easily as if the numbers were increasingly additive. Here is why. With 257 and the private key, we knew that there must be a 182 in it, because this was the largest number ≤ 257 . Without that increasingly additive property, looking at 192, can we say that 109 must be a part of the number?

Let us see what happens if we use the same decryption approach. So, we take 192 and subtract 109, giving us 83. The next largest number is 77, so we subtract 77 from 83, giving us 6. We subtract 6, giving us 0. Therefore, 192 is $109 + 77 + 6$, which gives us the message 01001010. However, this is not the original message (00011101).

We can still decrypt the message, but it just isn't as easy as we thought. Let us say we want to decrypt 192. We don't know what numbers to add up to give us 192, so we try all possibilities. We might try 31 by itself, $31 + 6$, $31 + 22$, $31 + 6 + 22$, and so forth. With 8 numbers, there are a total of $2^8 = 256$ combinations to try. Some of these 256 different combinations are shown in [Table 1.6](#). For a person, this might take a few minutes. For a computer, this will take a very small fraction of a second.

TABLE 1.6
Some Combinations of the Eight Public-Key Values

31 + 6	31 + 22	31 + 6 + 22	31 + 58	31 + 109	31 + 4
31 + 77	31 + 21	31 + 6 + 22	31 + 6 + 58	31 + 6 + 109	31 + 6 + 4
31 + 6 + 77	31 + 6 + 21	31 + 22 + 58	31 + 22 + 109	31 + 22 + 4	31 + 22 + 77
31 + 22 + 21	31 + 58 + 109	31 + 58 + 4	31 + 58 + 77	31 + 58 + 21	31 + 109 + 4
31 + 109 + 77	31 + 109 + 21	31 + 4 + 77	31 + 4 + 21	31 + 77 + 21	...
31 + 6 + 22 + 58	31 + 6 + 22 + 109	31 + 6 + 22 + 4	...	31 + 109 + 77 + 21	31 + 4 + 77 + 21
6 + 22	...	31 + 6 + 22 + 58 + 109 + 4 + 77		31 + 6 + 22 + 58 + 109 + 4 + 77 + 21	

For encryption to work, we need to ensure that not only can people not decrypt coded messages if they do not have the private key but also that computers cannot do so easily by trying all combinations of numbers. We therefore want to use a key whose size is larger than 8. What would be a good size for our encryption key? Frankly, the larger the better. If a key of 8 can be decrypted by a brute-force approach, as shown above, in 256 tries or less, what about a key of 10? 2^{10} is 1024, which will still be quite easy for a computer. What about a key of 20? There are 2^{20} different combinations of key values. This gives us a value a little more than 1 million combinations. Since modern personal computers operate at speeds of billions of instructions per second, even this key size is too weak. A key of 30 is too weak because 2^{30} is only around 1 billion. Supercomputers operate at speeds of trillions of operations per second, making 2^{40} and even 2^{50} and 2^{60} too weak. Instead, we prefer a key whose size is larger, perhaps 128 or even 256.

So, now, we see the types of encryption technologies available: public key and private key, with an option for the key's size. A number of algorithms are available to us, most of which are implemented in software so that we as users do not have to worry about the mathematics behind encryption and decryption. The Data Encryption Standard (DES), developed in the 1970s, is a private-key encryption algorithm that uses a 56-bit key size; this is too small for our needs today. The Advanced Encryption Standard (AES) is a follow-up algorithm that uses 128-bit key sizes instead. Triple DES is a variant of DES that uses three separate keys of 56 bits each. Although each key is only 56 bits, the combination of applying three keys makes it far harder to decrypt by a brute-force approach. Aside from these private-key encryption algorithms, there are also Message-Digest algorithms known as MDn, where n is a number, for example, MD5. These are 128-bit encryption algorithms that apply hash functions (division, saving the remainder similar to what we did when computing checksums). Another form of encryption uses the set of SHA: SHA-0, SHA-1, SHA-2, and SHA-3. SHA stands for *secure hash algorithm*. SHA operates similarly to MDn by converting a sequence of bits of a message into a hash value. SHA-0 was never utilized and SHA-1 was found to have security flaws, so SHA-2 is more commonly used when security is essential.

We can further enhance private-key encryption by utilizing a *nonce* value. This value is a random number (or pseudo-random number) generated one time for a single-encrypted communication. Once the sender has sent his or her message, the nonce value is not used again. With a nonce value, the encryption code becomes even harder to break because this particular code is used only one time, and so a collection of data that a decryption algorithm might try to analyze is not available. As another security precaution, the nonce value may come with a time stamp so that it can be applied only within a reasonable amount of time. Once that time limit elapses, the nonce value is no longer usable by the client attempting to encrypt a message.

For public-key encryption, there are several available algorithms. Wired Equivalent Privacy (WEP) was released in 1999; it used 26 10-digit values as keys. It has been replaced by both WPA and WPA2. WPA stands for Wi-Fi Protected Access, and as its name implies, it is used

for wireless communication. WPA2 replaced WPA, which, like SHA-1 and DES, was found to have significant security flaws. You might have used WPA2 to secure your own home wireless network. A feature known as Wi-Fi Protected Setup still has known flaws, and so it would be wise to avoid using this. Another public-key encryption algorithm is used in SSH, the Unix/Linux secure shell program. The specific algorithm used is denoted as SSH-1, SSH-2, or OpenSSH. The Secure Sockets Layer (SSL) and Transport Layer Security (TLS) are a pair of protocols that are combined to provide public-key encryption from within TCP/IP. We have noted that TCP/IP does not have its own means of security, so SSL/TLS has been added. These two protocols operate in the transport layer of TCP (the second from topmost layer). SSL/TLS can actually handle either public- or private-key encryption.

Finally, HTTP Secure (HTTPS) is a variation of the HTTP protocol used to obtain web pages from web servers. With HTTPS, we add a digital certificate, which is provided to the end user before secure communication begins. This certificate contains the public key as well as the information to ensure that the certificate belongs to the website or organization that it claims to belong to. If you connect to a web server using HTTPS and you do not get back a certificate, then you get an erroneous or out-of-date certificate. On the other hand, if you connect to a web server using HTTPS and you receive a certificate that is not signed, then you are warned that the site may not be trustworthy. If you agree to continue your communication with the site, or if the site's certificate is acceptable, then you have the public key available to encrypt your messages.

1.5.4 THE FIREWALL

The firewall, though not necessarily a component of a network, is critical for the security of the resources within a network. The firewall has become essential in today's world with our reliance on the Internet for e-commerce because there are a growing number of people who attempt to attack networks and their resources whether as a form of cyberwarfare, a criminal act, or simply to see what they can do.

The firewall can be software or hardware. As software, it is a program or operating system service running on a computer. As hardware, it is often a networked device positioned near or at the *edge* of the network. Routers and gateways, for instance, can serve as firewalls if programmed to do so. Note that most hardware-oriented firewalls will perform other network functions, and so in reality, the hardware firewall is software running on a non-computer networked device.

The firewall's job is to analyze every message coming in from the network or being sent out to the network to determine if that message should make it through the firewall. In this way, attacks from outside can be prevented if any given message *looks* suspicious. Outgoing messages can be prevented if, for instance, an organization does not wish certain types of messages to be sent (such as messages sent to a specific domain such as Facebook). See [Figure 1.15](#) for an illustration of a network firewall.

Firewalls are considered *filters*. Their role is to filter network messages so that only permissible messages are passed through. Put another way, the firewall filters out any messages deemed unnecessary or potentially hazardous. The firewall will consist of rules. Rules are in essence *if-then* statements. A rule, for instance, might say that *any message going out to Facebook will be blocked* or *any message coming in over a port that has not been opened will be blocked*. These rules can analyze just about any aspect of a message. Here is a list to illustrate some of the aspects that a rule can examine. Keep in mind that a rule can test any one or a combination of these attributes.

- Source address or port (incoming messages)
- Destination address or port (outgoing messages)
- Type of packet (UDP or TCP)
- The protocol of the packet (e.g., HTTP, FTP, and DNS)
- Interface connecting the device to the network (for a computer)

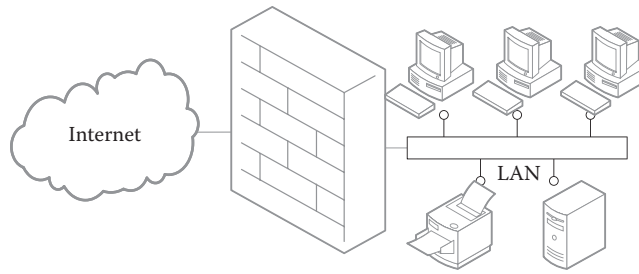


FIGURE 1.15 LAN with firewall.

- Size of message
- Time of day/day of week
- User who initiated the message (if available, this would require authentication)
- The state of the message (is this a new message, in response to another message etc.)
- The number of messages from this source (or to this destination)

In response to a rule being established, the firewall can do one of several things. It can allow the message to go through the firewall and onto the network or into the computer. It can reject the message. A rejection may result in an acknowledgment being sent back to the source so that the source does not continue to send the message (thinking that the destination has not acknowledged because it is not currently available). A rejection can also result in the message just being dropped without sending an acknowledgment. If the firewall is a network device (e.g., router), the message can be forwarded or rejected. Finally, the firewall could log the incident.

We are not covering more detail here because we need to better understand TCP/IP first. We will consider other forms of security as we move through the textbook, including using a proxy server to filter incoming web pages.

1.5.5 NETWORK CACHES

The word cache (pronounced *cash*) refers to a nearby stash of valuable items. We apply the word cache in many ways in computing. The most commonly used cache is a small amount of fast memory placed onto a central processing unit (CPU or processor). The CPU must access main memory to retrieve program instructions and data as it executes a program. Main memory (dynamic random-access memory [DRAM]) is a good deal slower than the CPU. If there was no faster form of memory, the CPU would continually have to pause to wait for DRAM to respond.

The CPU Cache, or SRAM (static RAM), is faster than DRAM and nearly as fast or equally as fast as the CPU's speed. By placing a cache on the processor, we can store portions of the program code and data there and will not have to resort to accessing DRAM unless what the CPU needs is not in cache. However, cache is more expensive, so we only place a small amount on the processor. This is an economic trade-off (as well as a space trade-off; the processor does not give us enough space to place a very large cache).

In fact, computers will often have several SRAM caches. First, each core (of a multicore CPU) will have its own internal instruction cache and internal data cache. We refer to these as L1 caches. In addition, both caches will probably also have a small cache known as the Translation Lookaside Buffer (TLB) to store paging information, used for handling virtual memory. Aside from the L1 caches, a processor may have a unified, larger L2 cache. Computers may also have an even larger L3 cache, either on the chip or off the chip on the motherboard.

The collection of caches, DRAM, and virtual memory (stored on hard disk) is known as the *memory hierarchy*. The CPU looks at the L1 cache first, and if the item is found, no further accessing is required. Otherwise, the CPU works its way down the memory hierarchy, until the item being sought is found: L2 cache, L3 cache, DRAM, and virtual memory. If an item is found lower in the hierarchy, it is copied into all higher levels along with its neighboring data locations. For instance, an item found at memory location X would be grouped into a block with items $X + 1$, $X + 2$, and $X + 3$. When X is found, X is moved up to the higher levels of the memory hierarchy along with the content found at $X + 1$, $X + 2$, and $X + 3$. Unfortunately, as you move up the memory hierarchy, less space is available, so moving something requires discarding something. A *cache replacement strategy* must be used to handle this situation.

Aside from a replacement strategy, we also have a cache issue known as *cache consistency*. If a shared datum (i.e., a datum that can be used by any of the cores at any time) has been moved into one core's L1 cache and later moved into another core's L1 cache, what happens if one core modifies the datum? The value that it has is *fresh*, whereas the other core's value in its cache is *dirty*. If we don't update the other cache before the datum is used by the other core, we have an out-of-date datum being used. This datum's appearance in memory is also outdated. If the first core brings something new to its L1 cache and chooses to discard the entry containing this modified datum, the datum needs to be saved to memory. The cache's *write policy* dictates whether this write will be performed at the time the datum is updated in cache or at the time the datum is discarded from cache.

Does the memory hierarchy have anything to do with networking? Not really! However, there are many caches that serve similar purposes to the L1, L2, and L3 caches. That is, by storing often-used data locally, we can reduce the amount of network communication required. Although network caches are often stored on hard disk, they do have two things in common with the L1/L2/L3 hardware caches: a need for a replacement strategy and the problem of cache consistency. The network caches do not typically need a write policy, because we are usually obtaining data from a server (a source or authority), and so our local computer will not be modifying the content.

We see many forms of network caches. Our web browsers have their own caches, whereby content of recently viewed web pages are stored. The web browser cache is stored on our computer's hard disk. A proxy server is used within an organization to store recently retrieved or accessed web content to be shared among all the clients within the network. When we visit a website, our web browser first looks at our local browser cache to see if the content is stored there. If so, the web browser loads the content from the local hard disk and does not communicate with the web server. If not, then the message sent from our web browser onto the Internet will be intercepted by our proxy server (if we have one). If the content is located there, it is returned to our browser, and our request does not proceed any further. There may also be caches located across the Internet on various servers and/or routers.

Another piece of data that can be cached is IP address to IP alias mapping (address resolution data). Our DNS handles the mapping for us. We use DNS name servers, which are distributed across the Internet. However, given the frequency that our computers perform address resolution, some common IP addresses/aliases could be cached on our local computer or a local server (our organization's DNS name server) to save time. These types of caches suffer from the same problems as the web browser/proxy server caches.

Aside from needing a replacement strategy for our caches should they become full, cache consistency is a problem. How does our local or proxy server cache know if the content has been updated?

In later chapters, we will revisit the idea of the DNS cache and the proxy server (note that a proxy server does more than cache content). We will also look at how we can control how long an item remains in a cache via server software for web servers, proxy servers, and name servers.

1.6 CHAPTER REVIEW

Terms introduced in this chapter are as follows:

- Anycast
- ARPANET
- Backbone
- Bluetooth
- Broadcast
- Cache memory
- Checksum
- Client
- Client–server network
- Computer network
- CRC
- DDN
- Encryption and decryption
- Encryption key
- Ethernet
- Firewall
- Frame Relay
- Gateway
- Hub
- Internet
- LAN
- MAC address
- MODEM
- Multicast
- Network cache
- Network device
- NIC
- Nonce value
- OSI model
- Packet switching
- Peer-to-peer network
- Private key
- Protocol
- Protocol stack
- Public key
- Repeater
- RFC
- Router
- Server
- Switch
- TCP/IP
- Topology
- Unicast
- WAN

REVIEW QUESTIONS

1. Define a network. How does a computer network differ from the more general term *network*?
2. What types of devices would you find in a typical computer network?
3. How does a hub differ from a switch? How does a switch differ from a router?
4. Of the gateway, hub, router, and switch, which is the most sophisticated form of network device?
5. A MAC table would be stored in which type of device, hub, router, or switch (or some combination)?
6. What information would you find in a routing table that you would not find in a network switch's table?
7. You want to send a signal to select devices in a LAN. Would you use anycast, broadcast, multicast, or unicast? Explain.
8. You want to send a signal to all devices in a LAN. Would you use anycast, broadcast, multicast, or unicast? Explain.
9. Which specific types of servers can be classified as file servers?
10. A proxy server is an intermediary between which two types of Internet devices?
11. Of twisted-wire pair, coaxial cable, and fiber optic cable, which one(s) is/are susceptible to electromagnetic interference?
12. Of twisted-wire pair, coaxial cable, and fiber optic cable, which one has the greatest bandwidth?
13. Of twisted-wire pair, coaxial cable, and fiber optic cable, which one has the most limited range, thus requiring repeaters to be used over distances of only a few hundred feet?
14. Both twisted-wire and coaxial cable are flexible. Is fiber optic cable flexible (e.g., bendable) or rigid?
15. You want to send a signal from your computer over the telephone network. Does the signal from your computer need to be modulated or demodulated? At the receiving end, before the other computer can use the signal, does it need to be modulated or demodulated?
16. How does an NIC differ from a MODEM?

17. In what way is the star topology more expensive than a bus or ring topology? Why is that cost not a dissuading factor when building networks today?
18. Of the bus, ring, and star topologies, which one necessarily becomes slower as the number of resources on the network increases?
19. Give an example of where you might find a PAN. Give an example of where you might find a CAN. Should we consider the PAN to be a LAN? Should we consider the CAN to be a LAN?
20. What is the role of the OSI's presentation layer?
21. At which layer of the OSI model does network addressing take place (i.e., at which layer are network addresses utilized?)
22. What is the difference between a layer 2 device and a layer 3 device?
23. Of TCP/IP and OSI, which is used as the basis for Internet communication?
24. Of TCP/IP and the OSI model, which one specifically proscribes a means of encryption?
25. What is X.25, and does it relate more to TCP/IP or OSI?
26. In X.25, what happens if a country has more than 10 different network addresses?
27. List five devices that use the Bluetooth technology. You might need to research this.
28. What has the Frame Relay protocol been used for, and is it still popular?
29. *True or false:* Alexander Graham Bell invented the first telephone.
30. *True or false:* Early computer networks were circuit-switched, but since the development of the ARPANET, computer networks have been packet-switched.
31. How many different computers were connected by the initial incarnation of the ARPANET? In which states were these computers located?
32. *True or false:* The original intention of the ARPANET was to provide emails between its users.
33. *True or false:* Before 1996, only Linux/Unix-based computers could connect to the Internet.
34. In which year was the World Wide Web created?
35. What is phishing?
36. How does the cache associated with the CPU differ from your web browser's cache? (Hint: Think about where the caches are physically stored.)
37. Define cache consistency.
38. List from fastest to slowest the parts of the memory hierarchy.

REVIEW PROBLEMS

1. Assume a signal frequency of 1 GHz (1 billion hertz). How many cycles (peak to valley to peak) can occur in 1 second?
2. If something has a frequency of 1 KHz (1000 hertz), what is its period?
3. You have a 200 GByte file. You want to transmit it over computer network. Using [Table 1.3](#), how long will it take to transmit the file using the maximum bandwidth, assuming that you are transmitting it over:
 - a. Unshielded twisted-wire pair
 - b. Fiber optic cable
 - c. Wireless using radio frequency
4. Approximately how much faster (or slower) is 4G bandwidth over: (i) wireless and (ii) 3G? (Use [Table 1.3](#).)
5. Assume that a network uses a 3D mesh topology. What is the minimum and maximum number of connections that any one device will have? (Refer to [Figure 1.9](#).)
6. Assume that you have 20 devices that you are connecting in a LAN using a full-mesh topology. How many total connections will each device have? How many total connections will be made to complete the network?
7. Using an ASCII table, compute an 8-bit checksum for the ASCII message *The Internet*. Remember to divide your sum by 256 and take the remainder.
8. Repeat #7 with the message "Information is not knowledge, knowledge is not wisdom."

9. Derive the CRC value for the binary value 110001110111100101110011 and the key 10000.
10. Repeat #9 with the key 10001.
Use the following eight integer numbers as a public key and answer questions 11–13.
1 4 6 13 25 52 110 251
11. Convert *Information* into ASCII and encrypt each byte.
12. Decrypt the following five integer numbers into a five-character ASCII string.
191 62 243 129 257
13. How many different combinations of sums are there with this key?

DISCUSSION QUESTIONS

1. Research a well-known MAN. How is that MAN different from a WAN? For instance, what specific applications does the MAN that you researched offer that the Internet does not?
2. Select an RFC and answer the following based on that RFC.
 - a. Summarize what the RFC was used for.
 - b. State the year in which the RFC was first offered.
 - c. Has the RFC been updated, and if so, when?
 - d. Has the RFC been superseded by a newer one? If so, when?
3. In your own words, state why Ethernet was a significant development in computer networks.
4. Why is it inappropriate to say that the Internet offers services? If the Internet does not offer us services, how are those services made available?
5. In your own words, explain what is packet switching. How does this differ from circuit switching?
6. What is the Internet backbone? Has there always been a single Internet backbone?
7. Research the SHA family of encryption algorithms. Compare SHA-0, SHA-1, SHA-2, and SHA-3. In your comparison, explain which of these is/are currently used and why the others are not.
8. If you are using a Unix/Linux system, identify the configuration file for your firewall and explore the rules. Attempt to explain what each rule specifies.



Taylor & Francis

Taylor & Francis Group

<http://taylorandfrancis.com>

2 Case Study: Building Local Area Networks

This chapter provides greater detail on some of the lower levels of a network protocol stack. It does so with a focus on construction of local area networks (LANs), both wired and wireless.

2.1 INTRODUCTION

The earliest form of computer networks was designed and deployed by the organizations that were going to use them. In the 1950s, companies with multiple mainframe computers may have wanted to have the mainframe computers communicate with each other. In 1949, the U.S. Air Force built the first MODEM (which stands for MODulation DEModulation), so that their computers could communicate over the telephone lines. Bell Telephones developed their own MODEMs to connect their collection of computers, called SAGE, in 1953. They later commercialized their MODEM in 1958.

The MODEM permitted long-distance communication between sites. What about an organization that had two or more mainframes on one site that they wished to connect directly together? The MODEM, with its slow speed, was not the answer. Instead, some form of direct cable could deliver faster communication while not requiring the use of telephone lines.

By the early 1960s, mainframe computer operating systems used either or both multiprogramming and multitasking. These terms express a computer's ability to run multiple programs *concurrently*. This means that the running processes are resident in memory and the computer's processor switches off quickly between them. The processor can run only one process at any moment in time, but because of its speed, switching off between multiple processes gives the illusion of running programs simultaneously.

In multiprogramming, a switch between processes occurs only if the current process needs time-consuming input or output. This is also known as *cooperative multitasking*. In *preemptive multitasking*, a switch between processes occurs when a preset time limit elapses, so that the processor can be shared among the multiple overlapping processes. Multiprogramming precedes preemptive multitasking, but any multitasking system is capable of both.

With the advent of concurrent processing operating systems, there was a need to connect multiple users to the mainframe, rather than the users coming to the computer center and standing in line, waiting to gain access to the computer. The approach taken was to equip each user with a simple input/output device called a *dumb terminal*. The input was through a keyboard, and the output was with a monitor, but the dumb terminal had no memory or processor. Instead, all commands would be sent from the terminal to the mainframe and any responses sent back to the terminal would be displayed. This led to a decentralized way to control the mainframe, or the first form of LAN. See [Figure 2.1](#).

In some cases, the connection between a dumb terminal and the mainframe (or other devices such as a line printer) would be a point-to-point connection. With a point-to-point connection, there was little need for some of the network overhead that is required of modern networks. For instance, a point-to-point network does not need source or destination addresses because any message is meant for the other end of the connection. However, in reality, most networks would contain a large number of dumb terminals, along with storage devices and printers. This would make a point-to-point network between each device and the mainframe prohibitively expensive, as the mainframe would need a port for each connection. This problem becomes even more exacerbated if the organization has multiple mainframes.